

xác suất cho các lớp khác). Việc tối thiểu hóa hàm chi phí được thể hiện trong **Phương trình 4-22**, được gọi là entropy chéo, sẽ dẫn đến mục tiêu này vì nó phạt mô hình khi nó ước tính xác suất thấp cho một lớp mục tiêu. Entropy chéo thường được sử dụng để đo lường mức độ phù hợp của một tập hợp các xác suất lớp ước tính với các lớp mục tiêu.

Phương trình 4-22. Hàm chi phí entropy chéo

$$J(\theta) = -\sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(\hat{p}^{(i)}(k))$$

Trong phương trình này:

- $\hat{p}^{(i)}(k)$ là xác suất mục tiêu mà trường hợp i thuộc về lớp k . Nói chung, nó bằng 1 hoặc 0, y_k tùy thuộc vào việc trường hợp đó có thuộc lớp hay không.

Lưu ý rằng khi chỉ có hai lớp ($K = 2$), hàm chi phí này tương đương với hàm chi phí của Hồi quy Logistic (mất mát logarit; xem **Phương trình 4-17**).

ENTROPY CHÉO

Khái niệm entropy chéo bắt nguồn từ lý thuyết thông tin của Claude Shannon . Giả sử bạn muốn truyền tải thông tin về thời tiết mỗi ngày một cách hiệu quả. Nếu có tám lựa chọn (nắng, mưa, v.v.), bạn có thể mã hóa mỗi lựa chọn bằng ba bit vì $2^3 = 8$. Tuy nhiên, nếu bạn nghĩ rằng trời sẽ nắng gần như mỗi ngày, thì sẽ hiệu quả hơn nhiều nếu mã hóa "nắng" chỉ bằng một bit (0) và bảy lựa chọn còn lại bằng bốn bit (bắt đầu từ 1). Entropy chéo đo lường số bit trung bình mà bạn thực sự gửi cho mỗi lựa chọn. Nếu giả định của bạn về thời tiết là hoàn hảo, entropy chéo sẽ bằng entropy của chính thời tiết (tức là tính không thể dự đoán được vốn có của nó). Nhưng nếu giả định của bạn sai (ví dụ, nếu trời mưa thường xuyên), entropy chéo sẽ lớn hơn một lượng được gọi là độ phân kỳ Kullback-Leibler (KL).

Độ lệch entropy chéo giữa hai phân phối xác suất p và q được định nghĩa là $H(p,q) = -\sum p(x) \log q(x)$ (ít nhất là khi các phân phối là rời rạc). Để biết thêm chi tiết, hãy xem [video của tôi về chủ đề này](#).

Vectơ đạo hàm của hàm chi phí này đối với θ được cho bởi **Phương trình 4-23**.

Phương trình 4-23. Vectơ độ dốc entropy chéo cho lớp k

$$\nabla_{\theta} J^{(k)}(\theta) = \sum_{i=1}^m (p^{(i)}(k) - y_k^{(i)}) x^{(i)}$$

Giờ đây, bạn có thể tính toán vectơ gradient cho mỗi lớp, sau đó sử dụng thuật toán Gradient Descent (hoặc bất kỳ thuật toán tối ưu hóa nào khác) để tìm ma trận tham số θ sao cho hàm chi phí được tối thiểu hóa.

Hãy sử dụng Softmax Regression để phân loại cây hoa diên vĩ vào cả ba lớp. LogisticRegression của Scikit-Learn tự động sử dụng Softmax Regression khi bạn huấn luyện nó trên nhiều hơn hai lớp (giả sử bạn sử dụng solver="lbfgs", đây là mặc định). Nó cũng áp dụng chuẩn hóa ℓ theo mặc định, mà bạn có thể kiểm soát bằng siêu tham số C , như đã nói ở trên:

```
X = iris.data[["chiều dài cánh hoa (cm)", "chiều rộng cánh hoa (cm)"]].values
y = iris["mục tiêu"]
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

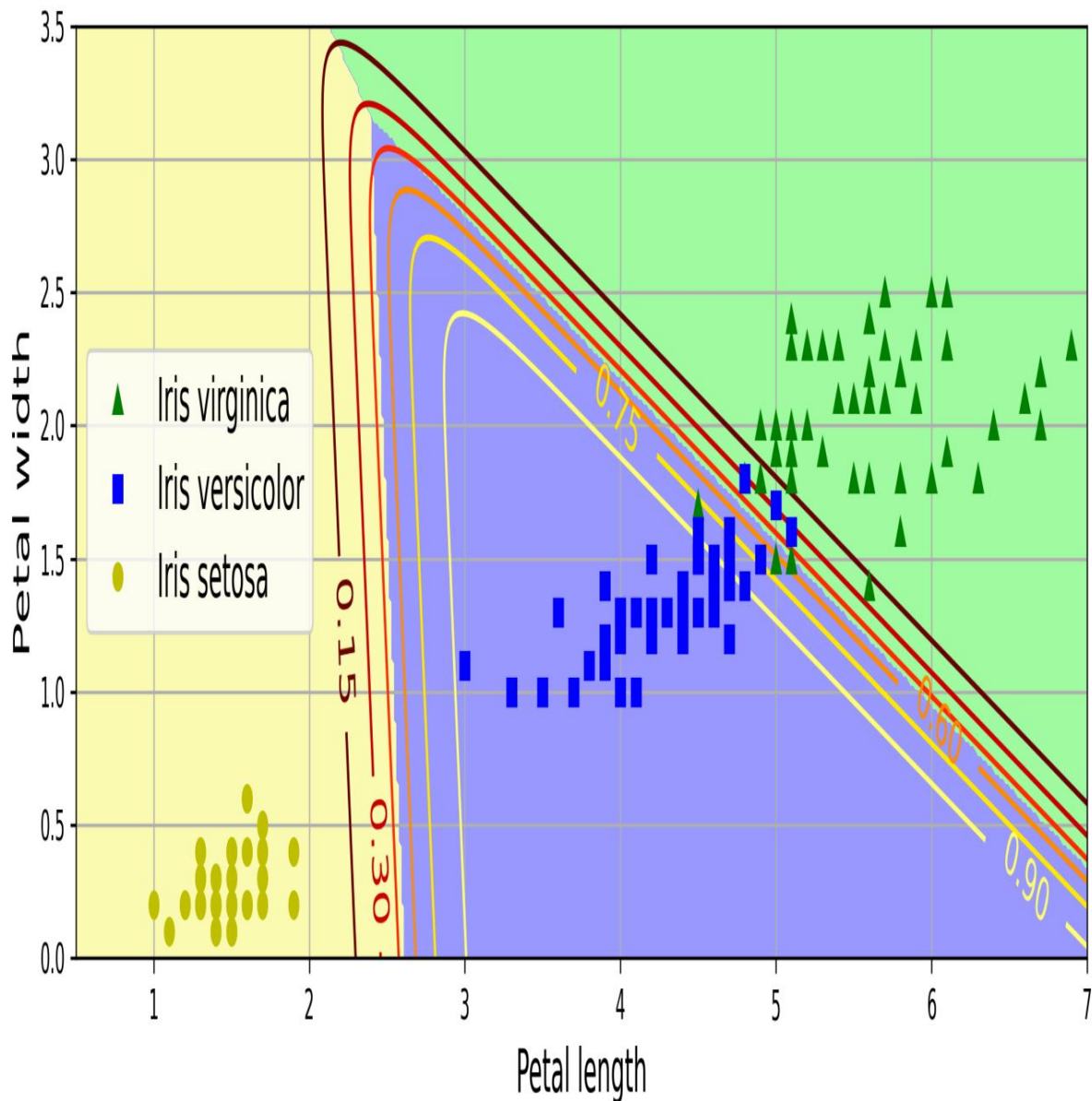
softmax_reg = LogisticRegression(C=30, random_state=42)
softmax_reg.fit(X_train, y_train)
```

Vậy nên lần tới khi bạn thấy một bông hoa diên vĩ có cánh dài 5 cm và rộng 2 cm, bạn có thể hỏi mô hình của mình xem đó là loại diên vĩ nào, và nó sẽ trả lời là Iris virginica (loại 2) với xác suất 96% (hoặc Iris versicolor) .

(với xác suất 4%)

```
>>> softmax_reg.predict([[5, 2]]) array([2])
>>> softmax_reg.predict_proba([[5, 2]]).round(2) array([[0. , 0.04,
0.96]])
```

Hình 4-25 hiển thị các ranh giới quyết định thu được, được biểu thị bằng màu nền. Lưu ý rằng các ranh giới quyết định giữa bất kỳ hai lớp nào đều là đường thẳng. Hình cũng hiển thị xác suất cho lớp Iris versicolor, được biểu thị bằng các đường cong (ví dụ: đường có nhãn 0,30 biểu thị ranh giới xác suất 30%). Lưu ý rằng mô hình có thể dự đoán một lớp có xác suất ước tính dưới 50%. Ví dụ, tại điểm mà tất cả các ranh giới quyết định gặp nhau, tất cả các lớp đều có xác suất ước tính bằng nhau là 33%.



Hình 4-25. Ranh giới quyết định của hồi quy Softmax

Trong chương này, bạn đã học được nhiều cách khác nhau để huấn luyện các mô hình tuyến tính, cả cho hồi quy và phân loại. Bạn đã sử dụng phương trình dạng đóng để giải bài toán Hồi quy tuyến tính, cũng như thuật toán Giảm độ dốc, và bạn đã học được cách thức hoạt động của nhiều phương pháp khác nhau.

Các hình phạt có thể được thêm vào hàm chi phí trong quá trình huấn luyện để điều chỉnh mô hình. Trong quá trình đó, bạn cũng đã học cách vẽ đồ thị đường cong học tập và phân tích chúng, cũng như cách thực hiện dừng sớm. Cuối cùng, bạn đã học cách hoạt động của Hồi quy Logistic và Hồi quy Softmax. Chúng ta đã mở ra những "hộp đen" đầu tiên của Học máy! Trong các chương tiếp theo, chúng ta sẽ mở ra nhiều "hộp đen" hơn nữa, bắt đầu với Máy vectơ hỗ trợ (SVM).

Bài tập

1. Bạn có thể sử dụng thuật toán huấn luyện hồi quy tuyến tính nào nếu bạn có một tập dữ liệu huấn luyện với hàng triệu dữ liệu đặc trưng?
2. Giả sử các đặc trưng trong tập dữ liệu huấn luyện của bạn có thang đo rất khác nhau. Những thuật toán nào có thể gặp khó khăn khi...
Điều này xảy ra như thế nào? Bạn có thể làm gì để giải quyết vấn đề này?
3. Thuật toán Gradient Descent có thể bị kẹt ở điểm cực tiểu cục bộ khi huấn luyện mô hình Hồi quy Logistic không?
4. Liệu tất cả các thuật toán Gradient Descent có dẫn đến cùng một mô hình nếu bạn cho chúng chạy đủ lâu?
5. Giả sử bạn sử dụng thuật toán Batch Gradient Descent và vẽ biểu đồ lỗi xác thực ở mỗi epoch. Nếu bạn nhận thấy lỗi xác thực liên tục tăng lên, điều gì có thể đang xảy ra? Làm thế nào bạn có thể khắc phục điều này?
6. Liệu có nên dừng thuật toán Mini-batch Gradient Descent ngay lập tức khi lỗi xác thực tăng lên không?
7. Trong số các thuật toán Gradient Descent mà chúng ta đã thảo luận, thuật toán nào sẽ đạt đến vùng lân cận của lời giải tối ưu nhanh nhất? Thuật toán nào sẽ thực sự hội tụ? Làm thế nào để các thuật toán khác cũng hội tụ?
8. Giả sử bạn đang sử dụng hồi quy đa thức. Bạn vẽ đồ thị đường cong học tập và nhận thấy có một khoảng cách lớn giữa lỗi huấn luyện và lỗi kiểm định. Điều gì đang xảy ra? Có ba cách nào để giải quyết vấn đề này?
9. Giả sử bạn đang sử dụng hồi quy Ridge và nhận thấy rằng lỗi huấn luyện và lỗi kiểm định gần như bằng nhau và khá cao. Bạn có cho rằng mô hình bị sai lệch cao hay phương sai cao?
Bạn nên tăng hay giảm siêu tham số điều chỉnh α ?
10. Tại sao bạn lại muốn sử dụng:
 - a. Sử dụng hồi quy Ridge thay vì hồi quy tuyến tính thông thường (tức là không sử dụng bất kỳ phương pháp điều chỉnh nào)?
 - b. Sử dụng Lasso thay vì hồi quy Ridge?
 - c. Sử dụng lưới đàn hồi thay vì dây thông lỏng?
11. Giả sử bạn muốn phân loại ảnh thành ảnh ngoài trời/trong nhà và ban ngày/ban đêm. Bạn nên thực hiện điều gì?
Nên dùng hai bộ phân loại Hồi quy Logistic hay một bộ phân loại Hồi quy Softmax?
12. Triển khai thuật toán Batch Gradient Descent với dừng sớm cho hồi quy Softmax mà không sử dụng Scikit-Learn, chỉ sử dụng NumPy.
Áp dụng nó vào một bài toán phân loại, ví dụ như tập dữ liệu iris.

Đáp án cho các bài tập này có sẵn ở cuối tập tài liệu của chương này, tại địa chỉ <https://homl.info/colab3>.

1. Một phương trình dạng đóng chỉ bao gồm một số hữu hạn các hằng số, biến số và các phép toán chuẩn, ví dụ: $a = \sin(b - c)$. Không có Tổng vô hạn, không giới hạn, không tích phân, v.v.

2. Về mặt kỹ thuật, đạo hàm của nó là hàm liên tục Lipschitz.

3. Vì đặc điểm 1 nhỏ hơn, nên cần một sự thay đổi lớn hơn ở θ để ảnh hưởng đến hàm chi phí, đó là lý do tại sao bất bị kéo dài dọc theo trục θ .

4. Eta (η) là chữ cái thứ bảy trong bảng chữ cái Hy Lạp.

5. Trong khi phương trình chuẩn chỉ có thể thực hiện hồi quy tuyến tính, các thuật toán Gradient Descent có thể được sử dụng để huấn luyện nhiều mô hình khác, như...

Chúng ta hãy chờ xem.

1

- 6. Khái niệm về độ lệch này không nên bị nhầm lẫn với thuật ngữ độ lệch của các mô hình tuyến tính.
- 7. Người ta thường sử dụng ký hiệu $J(\theta)$ cho các hàm chi phí không có tên viết tắt; chúng ta sẽ thường xuyên sử dụng ký hiệu này trong suốt phần còn lại của tài liệu này. Cuốn sách. Ngữ cảnh sẽ làm rõ hàm chi phí nào đang được thảo luận.
- 8. Các chuẩn mực được thảo luận trong [Chương 2](#).
- 9. Một ma trận vuông toàn số 0 ngoại trừ các số 1 trên đường chéo chính (từ trên cùng bên trái xuống dưới cùng bên phải).
- 10. Ngoài ra, bạn cũng có thể sử dụng lớp Ridge với bộ giải "sag". Stochastic Average GD là một biến thể của Stochastic GD. Để biết thêm chi tiết, Xem bài thuyết trình ["Giảm thiểu tổng hữu hạn bằng thuật toán gradient trung bình ngẫu nhiên"](#) bởi Mark Schmidt và cộng sự từ Đại học British Columbia.
- 11. Bạn có thể coi vectơ đạo hàm riêng tại một điểm không khả vi như một vectơ trung gian giữa các vectơ đạo hàm xung quanh điểm đó.
- 12. Bức ảnh được sao chép từ các trang Wikipedia tương ứng. Ảnh hoa diên vĩ (Iris virginica) của Frank Mayfield ([Creative Commons BY-SA 2.0](#)), Hoa diên vĩ Ảnh versicolor của D. Gordon E. Robertson ([Creative Commons BY-SA 3.0](#)), Ảnh Iris setosa miền công cộng.
- 13. Hàm reshape() của NumPy cho phép một chiều có giá trị là -1, nghĩa là "tự động": giá trị được suy ra từ độ dài của mảng. và các kích thước còn lại.
- 14. Tập hợp các điểm x sao cho $\theta^T x + \theta_0 x = 0$ xác định một đường thẳng.

Chương 5. Máy hỗ trợ vectơ

Với sách điện tử phát hành sớm, bạn sẽ nhận được sách ở dạng sơ khai nhất—nội dung thô và chưa chỉnh sửa của tác giả khi họ viết—nhờ đó bạn có thể tận dụng những công nghệ này rất lâu trước khi các tựa sách được phát hành chính thức.

Đây sẽ là chương thứ 5 của cuốn sách cuối cùng. Kho lưu trữ GitHub là <https://github.com/ageron/handson-ml3>.

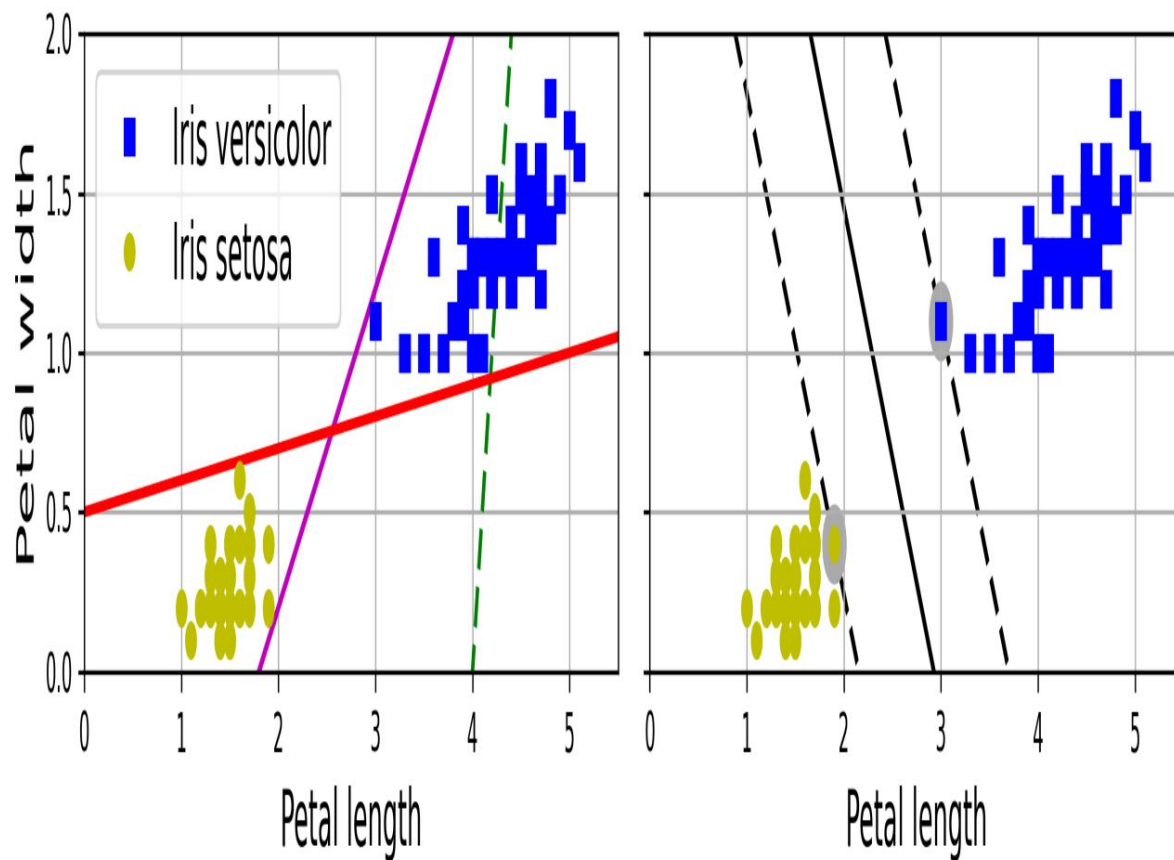
Nếu bạn có ý kiến đóng góp về cách chúng tôi có thể cải thiện nội dung và/hoặc ví dụ trong cuốn sách này, hoặc nếu bạn nhận thấy thiếu sót nào trong chương này, vui lòng liên hệ với biên tập viên theo địa chỉ mcroron@oreilly.com.

Máy hỗ trợ vectơ (SVM) là một mô hình học máy mạnh mẽ và linh hoạt, có khả năng thực hiện phân loại tuyến tính hoặc phi tuyến tính, hồi quy và thậm chí cả phát hiện sự mới lạ. SVM hoạt động hiệu quả với các tập dữ liệu phi tuyến tính có kích thước nhỏ đến trung bình (tức là hàng trăm đến hàng nghìn trường hợp), đặc biệt là đối với các nhiệm vụ phân loại. Tuy nhiên, chúng không mở rộng tốt lắm đối với các tập dữ liệu rất lớn, như chúng ta sẽ thấy.

Chương này sẽ giải thích các khái niệm cốt lõi của SVM, cách sử dụng chúng và cách chúng hoạt động. Chúng ta cùng bắt đầu nào!

Phân loại SVM tuyến tính

Ý tưởng cơ bản đằng sau SVM được giải thích rõ nhất bằng hình ảnh. **Hình 5-1** hiển thị một phần của tập dữ liệu iris đã được giới thiệu ở cuối **Chương 4**. Hai lớp có thể được phân tách rõ ràng dễ dàng bằng một đường thẳng (chúng có thể phân tách tuyến tính). Biểu đồ bên trái hiển thị ranh giới quyết định của ba bộ phân loại tuyến tính khả thi. Mô hình có ranh giới quyết định được biểu thị bằng đường nét đứt quá tệ đến nỗi nó thậm chí không phân tách các lớp một cách chính xác. Hai mô hình còn lại hoạt động hoàn hảo trên tập huấn luyện này, nhưng ranh giới quyết định của chúng quá gần với các trường hợp dữ liệu nên các mô hình này có thể sẽ không hoạt động tốt trên các trường hợp dữ liệu mới. Ngược lại, đường liền nét trong biểu đồ bên phải biểu thị ranh giới quyết định của bộ phân loại SVM; đường này không chỉ phân tách hai lớp mà còn giữ khoảng cách càng xa các trường hợp huấn luyện gần nhất càng tốt. Bạn có thể hình dung bộ phân loại SVM như việc tạo ra con đường rộng nhất có thể (được biểu thị bằng các đường nét đứt song song) giữa các lớp. Điều này được gọi là phân loại biên độ lớn.

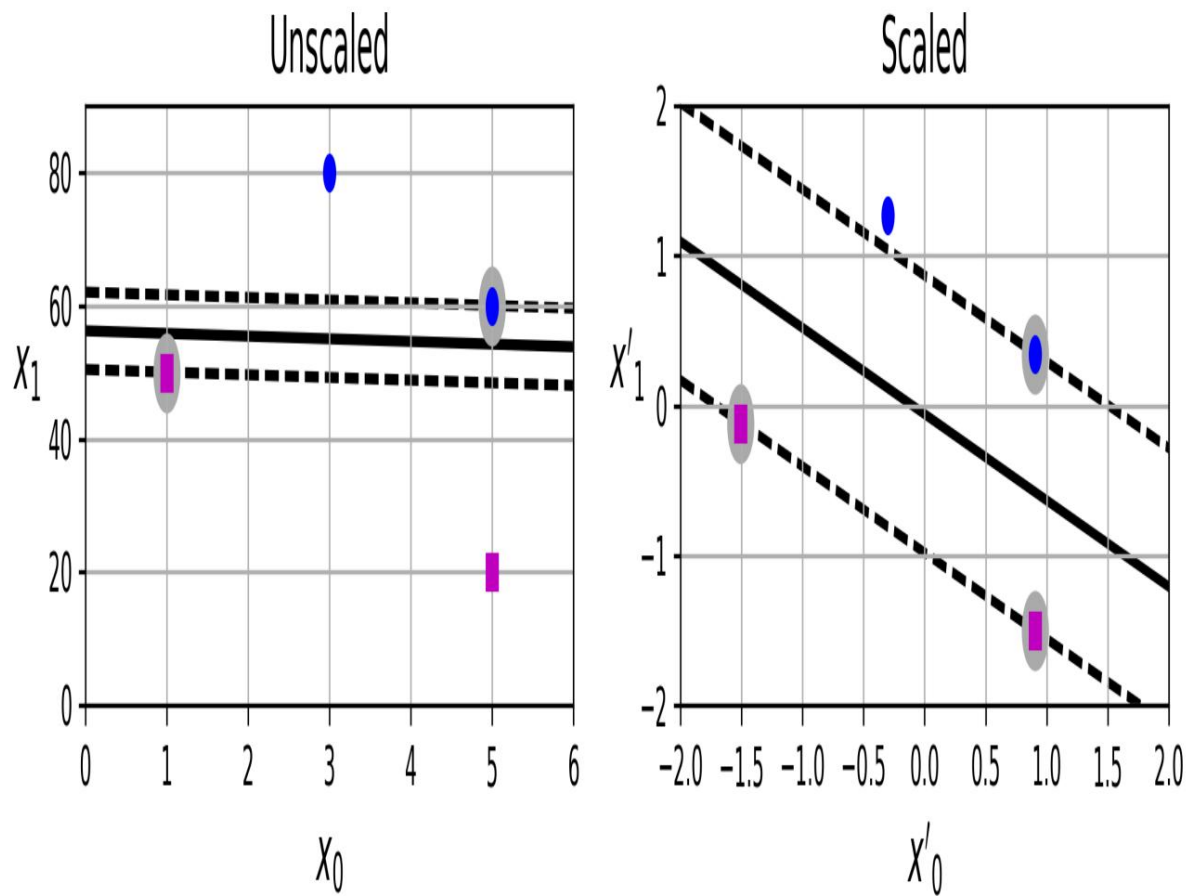


Hình 5-1. Phân loại lề lớn

Lưu ý rằng việc thêm nhiều trường hợp huấn luyện "ngoài đường" sẽ không ảnh hưởng đến ranh giới quyết định: ranh giới này hoàn toàn được xác định (hoặc "được hỗ trợ") bởi các trường hợp nằm ở rìa đường. Các trường hợp này được gọi là vectơ hỗ trợ (chúng được khoanh tròn trong [Hình 5-1](#)).

CẢNH BÁO

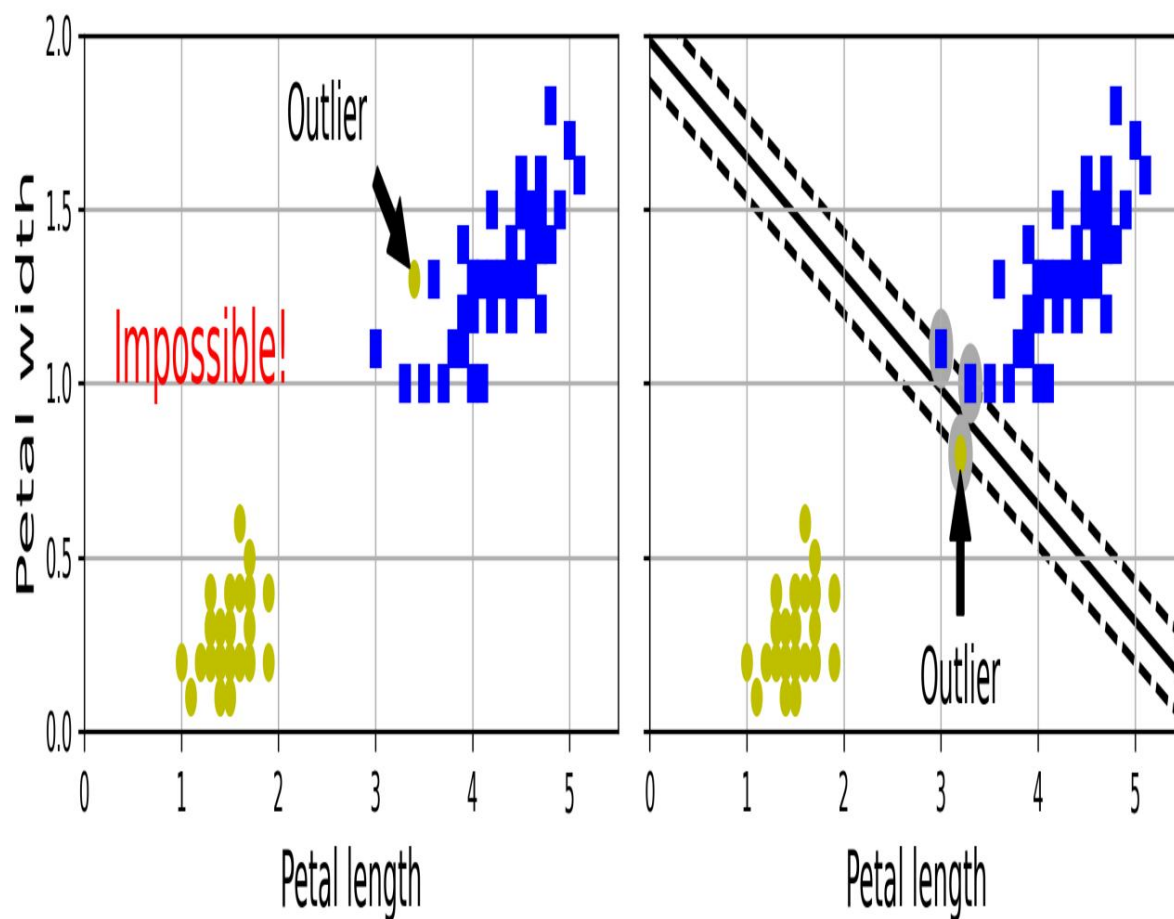
SVM rất nhạy cảm với tỷ lệ kích thước đặc trưng, như bạn có thể thấy trong [Hình 5-2](#): trong biểu đồ bên trái, tỷ lệ trục dọc lớn hơn nhiều so với tỷ lệ trục ngang, do đó con đường rộng nhất có thể gần như nằm ngang. Sau khi chuẩn hóa kích thước đặc trưng (ví dụ: sử dụng `StandardScaler` của `Scikit-Learn`), ranh giới quyết định trong biểu đồ bên phải trông tốt hơn nhiều.



Hình 5-2. Độ nhạy cảm với thang đo đặc trưng

Phân loại lề mềm

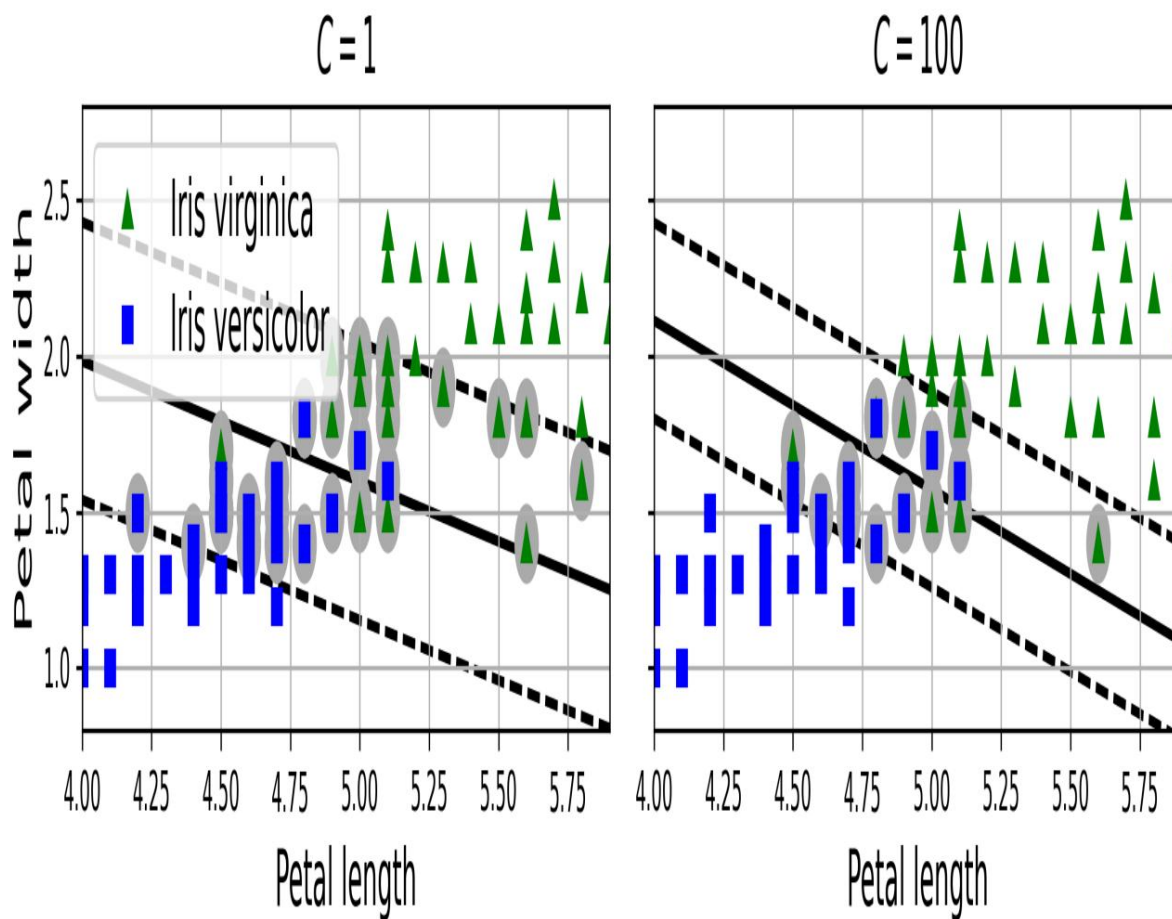
Nếu chúng ta quy định nghiêm ngặt rằng tất cả các trường hợp phải nằm ngoài đường phỏ và ở đúng phía, thì đây được gọi là phân loại ranh giới cứng. Có hai vấn đề chính với phân loại ranh giới cứng. Thứ nhất, nó chỉ hoạt động nếu dữ liệu có thể phân tách tuyến tính. Thứ hai, nó nhạy cảm với các giá trị ngoại lai. Hình 5-3 cho thấy tập dữ liệu iris chỉ với một giá trị ngoại lai bổ sung: ở bên trái, không thể tìm thấy ranh giới cứng; ở bên phải, ranh giới quyết định cuối cùng rất khác so với ranh giới mà chúng ta đã thấy trong Hình 5-1 khi không có giá trị ngoại lai, và có lẽ nó sẽ không khái quát hóa tốt.



Hình 5-3. Độ nhạy của biên độ cứng đối với các giá trị ngoại lệ

Để tránh những vấn đề này, hãy sử dụng mô hình linh hoạt hơn. Mục tiêu là tìm ra sự cân bằng tốt giữa việc giữ cho đường phố càng rộng càng tốt và hạn chế các vi phạm lề đường (tức là các trường hợp xe cộ đi vào giữa đường hoặc thậm chí sang sai làn đường). Điều này được gọi là phân loại lề đường mềm.

Khi tạo mô hình SVM bằng Scikit-Learn, bạn có thể chỉ định một số siêu tham số, bao gồm một siêu tham số điều chỉnh gọi là C . Nếu bạn đặt giá trị thấp, bạn sẽ có mô hình ở bên trái Hình 5-4. Với giá trị cao, bạn sẽ có mô hình ở bên phải. Như bạn thấy, giảm C làm cho "đường phố" lớn hơn, nhưng cũng dẫn đến nhiều vi phạm lề hơn. Nói cách khác, giảm C dẫn đến nhiều trường hợp hỗ trợ "đường phố", do đó ít có nguy cơ quá khớp. Nhưng nếu bạn giảm nó quá nhiều, mô hình sẽ bị thiếu khớp, như trường hợp ở đây: mô hình với $C=100$ có vẻ sẽ tổng quát hóa tốt hơn mô hình với $C=1$.



Hình 5-4. Lệ lớn (trái) so với ít vi phạm lệ hơn (phải)

MẸO

Nếu mô hình SVM của bạn bị quá khớp, bạn có thể thử điều chỉnh nó bằng cách giảm C .

Đoạn mã Scikit-Learn sau đây tải tập dữ liệu iris và huấn luyện bộ phân loại SVM tuyến tính để phát hiện hoa Iris virginica .

Quy trình này trước tiên chuẩn hóa các đặc trưng, sau đó sử dụng LinearSVC với $C=1$:

```
from sklearn.datasets import load_iris from
sklearn.pipeline import make_pipeline from
sklearn.preprocessing import StandardScaler from sklearn.svm
import LinearSVC

iris = load_iris(as_frame=True)
X = iris.data[["chiều dài cánh hoa (cm)", "chiều rộng cánh hoa (cm)"].values y =
(iris.target == 2) # Iris virginica

svm_clf = make_pipeline(StandardScaler(),
                        LinearSVC(C=1, random_state=42))
svm_clf.fit(X, y)
```

Mô hình thu được được thể hiện ở bên trái trong Hình 5-4.

Sau đó, như thường lệ, bạn có thể sử dụng mô hình để đưa ra dự đoán:

```
>>> X_new = [[5.5, 1.7], [5.0, 1.5]] >>>
svm_clf.predict(X_new)
```

```
mảng([ Đúng, Sai])
```

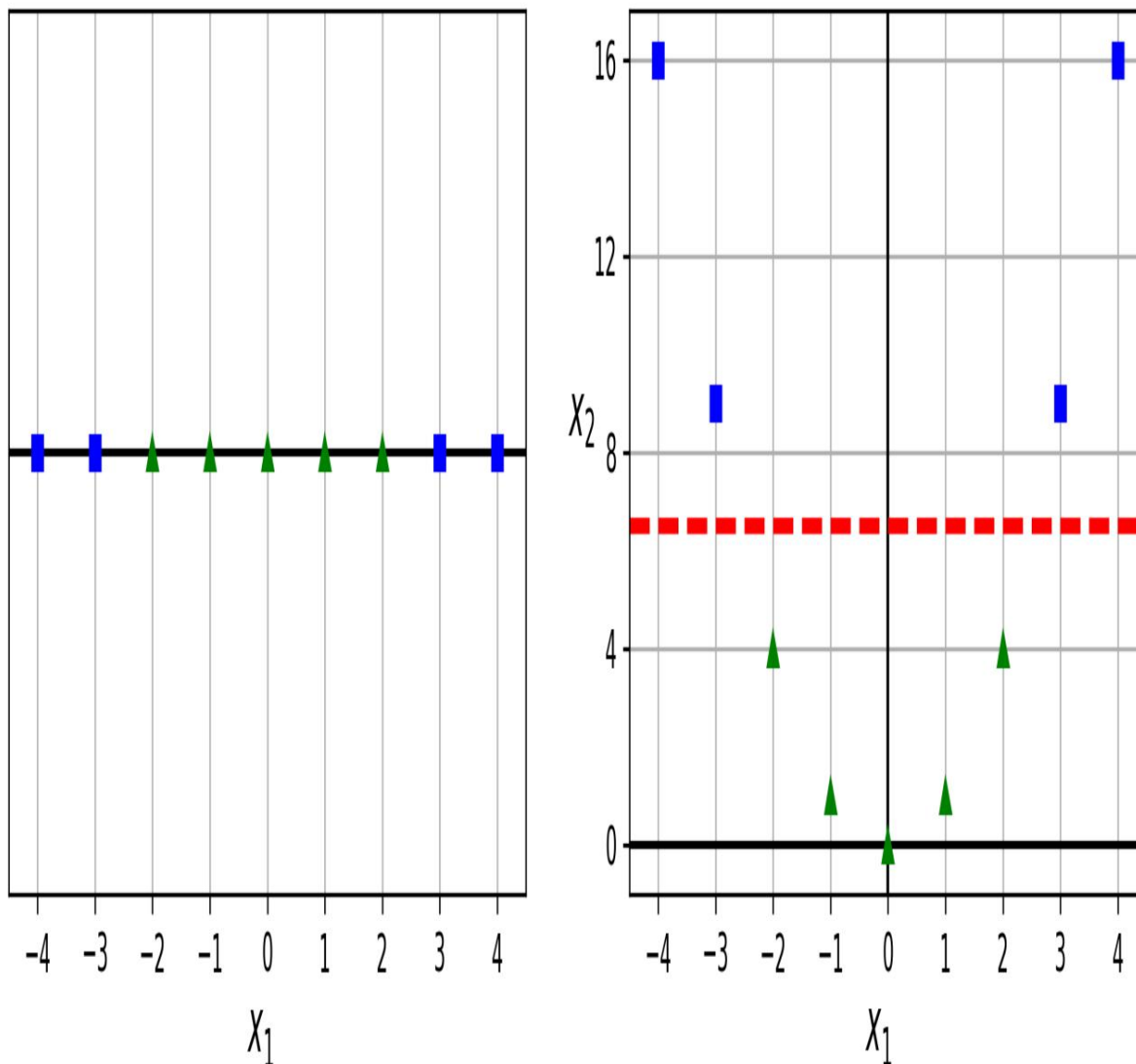
Cây đầu tiên được phân loại là Iris virginia, trong khi cây thứ hai thì không. Hãy xem xét các điểm số mà SVM đã sử dụng để đưa ra những dự đoán này. Chúng đo khoảng cách có dấu giữa mỗi trường hợp và ranh giới quyết định:

```
>>> svm_clf.decision_function(X_new)
array([ 0.66163411, -0.22036063])
```

Không giống như bộ phân loại LogisticRegression, LinearSVC không có phương thức predict_proba() để ước tính xác suất lớp. Tuy nhiên, nếu bạn sử dụng lớp SVC (sẽ được thảo luận sau) thay vì LinearSVC, và nếu bạn đặt xác suất của nó thành True, thì mô hình sẽ huấn luyện thêm một mô hình khác ở cuối quá trình huấn luyện để ánh xạ điểm số của hàm quyết định SVM sang xác suất ước tính. Về cơ bản, điều này yêu cầu sử dụng phương pháp kiểm định chéo 5 lần để tạo ra các dự đoán ngoài mẫu cho mỗi trường hợp trong tập huấn luyện, sau đó huấn luyện một mô hình LogisticRegression, do đó nó sẽ làm chậm quá trình huấn luyện đáng kể. Sau đó, các phương thức predict_proba() và predict_log_proba() sẽ khả dụng.

Phân loại SVM phi tuyến tính

Mặc dù các bộ phân loại SVM tuyến tính hiệu quả và thường hoạt động tốt một cách đáng ngạc nhiên, nhưng nhiều tập dữ liệu thậm chí không gần với việc phân tách tuyến tính. Một cách tiếp cận để xử lý các tập dữ liệu phi tuyến tính là thêm nhiều đặc trưng hơn, chẳng hạn như các đặc trưng đa thức (như bạn đã làm trong [Chương 4](#)); trong một số trường hợp, điều này có thể dẫn đến một tập dữ liệu phân tách tuyến tính. Hãy xem xét biểu đồ bên trái trong [Hình 5-5](#): nó biểu diễn một tập dữ liệu đơn giản chỉ với một đặc trưng, x_1 . Tập dữ liệu này không thể phân tách tuyến tính, vì bạn có thể thấy. Nhưng nếu bạn thêm một đặc trưng thứ hai $x_2 = (x_1)^2$, thì tập dữ liệu 2D thu được hoàn toàn có thể phân tách tuyến tính.



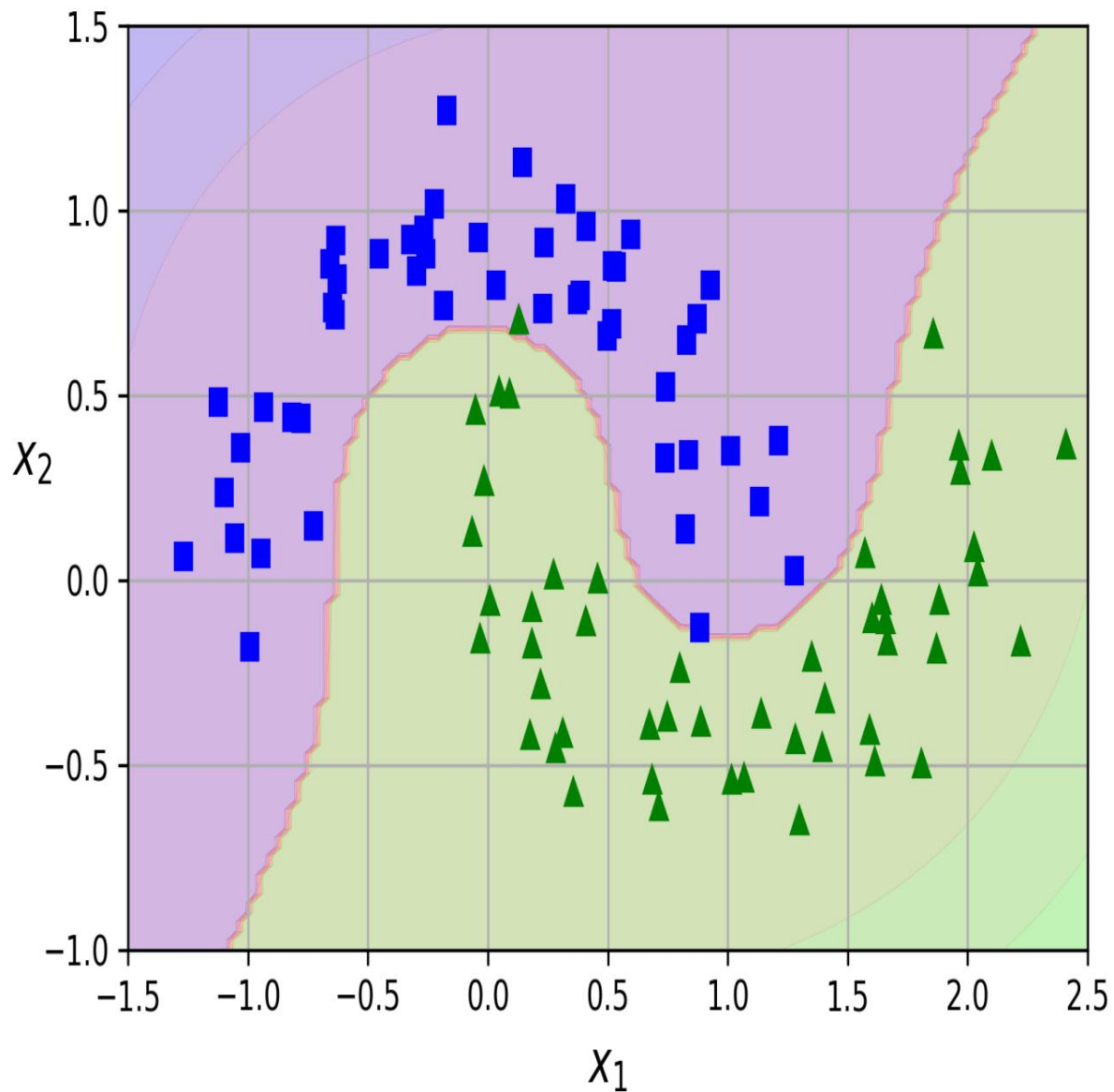
Hình 5-5. Thêm các tính năng để làm cho tập dữ liệu có thể phân tách tuyến tính.

Để triển khai ý tưởng này bằng Scikit-Learn, hãy tạo một pipeline chứa transformer `PolynomialFeatures` (đã thảo luận trong phần “[Hồi quy đa thức](#)”), tiếp theo là `StandardScaler` và `LinearSVC`. Chúng ta hãy thử nghiệm điều này trên tập dữ liệu moons: đây là một tập dữ liệu mẫu cho phân loại nhị phân, trong đó các điểm dữ liệu có hình dạng hai hình trăng lưỡi liềm xen kẽ nhau (xem [Hình 5-6](#)). Bạn có thể tạo tập dữ liệu này bằng cách sử dụng hàm `make_moons()`:

```
from sklearn.datasets import make_moons
from sklearn.preprocessing import PolynomialFeatures

X, y = make_moons(n_samples=100, noise=0.15, random_state=42)

polynomial_svm_clf = make_pipeline(
    Đặc trưng đa thức (bậc = 3),
    StandardScaler(),
    LinearSVC(C=10, max_iter=10_000, random_state=42)
) polynomial_svm_clf.fit(X, y)
```



Hình 5-6. Bộ phân loại SVM tuyến tính sử dụng các đặc trưng đa thức

Hạt nhân đa thức

Việc thêm các đặc trưng đa thức rất dễ thực hiện và có thể hoạt động tốt với nhiều thuật toán Học máy khác nhau (không chỉ SVM).

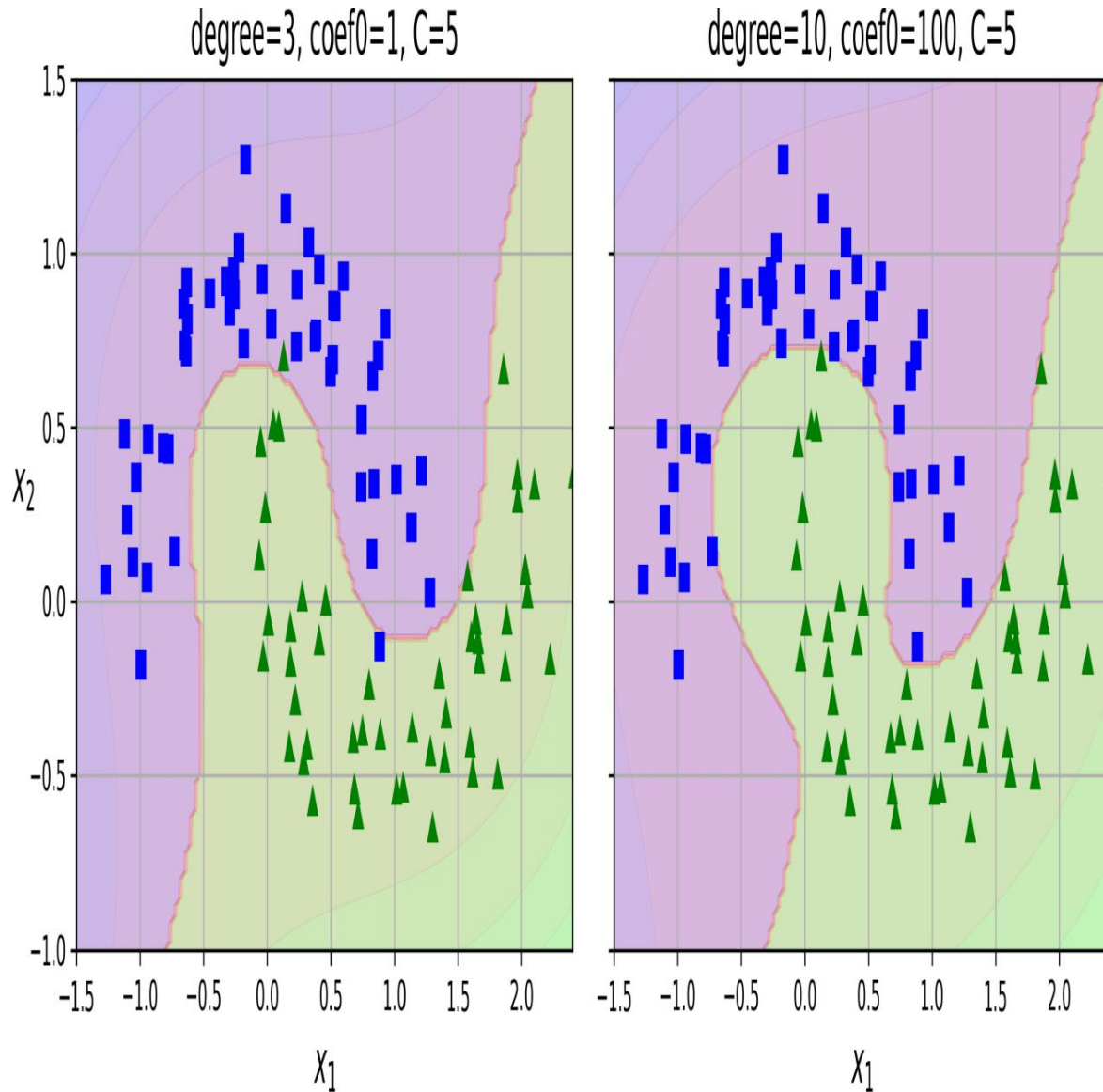
Tuy nhiên, ở bậc đa thức thấp, phương pháp này không thể xử lý các tập dữ liệu rất phức tạp, còn ở bậc đa thức cao, nó tạo ra một số lượng lớn các đặc trưng, khiến mô hình hoạt động quá chậm.

May mắn thay, khi sử dụng SVM, bạn có thể áp dụng một kỹ thuật toán học gần như kỳ diệu được gọi là thủ thuật kernel (sẽ được giải thích chi tiết hơn ở phần sau của chương này). Thủ thuật kernel cho phép thu được kết quả tương tự như khi bạn cộng nhiều đặc trưng đa thức, thậm chí với bậc rất cao, mà không cần thực sự cộng chúng. Điều này có nghĩa là không có sự bùng nổ tổ hợp về số lượng đặc trưng. Thủ thuật này được triển khai bởi lớp SVC. Hãy thử nghiệm nó trên tập dữ liệu moons:

```
from sklearn.svm import SVC

poly_kernel_svm_clf = make_pipeline(StandardScaler(),
                                     SVC(kernel="poly", degree=3, coef0=1, C=5))
poly_kernel_svm_clf.fit(X, y)
```

Đoạn mã này huấn luyện một bộ phân loại SVM sử dụng hạt nhân đa thức bậc ba. Nó được thể hiện ở bên trái trong Hình 5-7. Bên phải là một bộ phân loại SVM khác sử dụng hạt nhân đa thức bậc 10. Rõ ràng, nếu mô hình của bạn bị quá khớp (overfitting), bạn có thể muốn giảm bậc đa thức. Ngược lại, nếu nó bị thiếu khớp (underfitting), bạn có thể thử tăng bậc đa thức. Siêu tham số coef0 kiểm soát mức độ ảnh hưởng của các số hạng bậc cao so với các số hạng bậc thấp đến mô hình.



Hình 5-7. Bộ phân loại SVM với nhân đa thức

MẸO

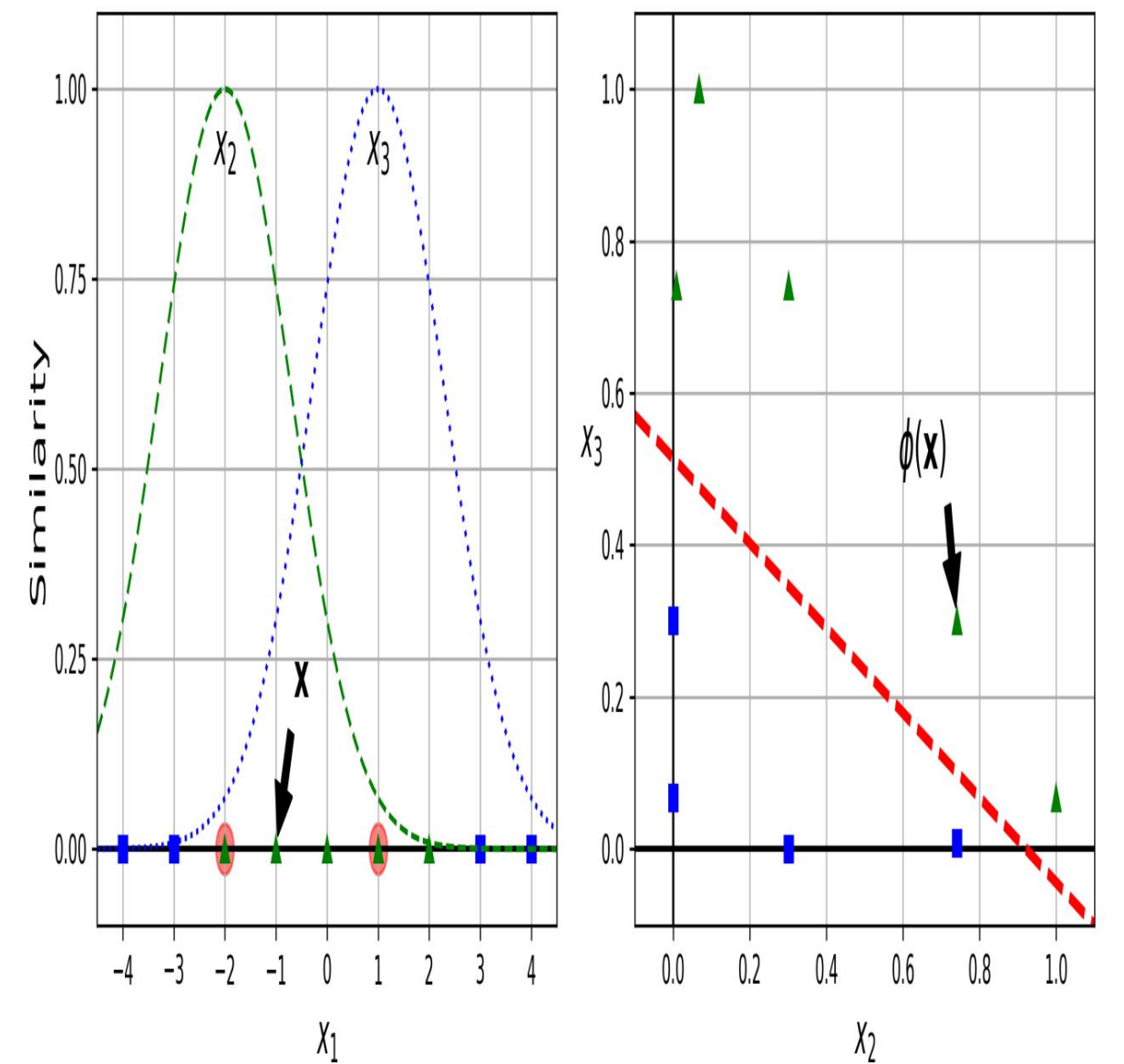
Mặc dù các siêu tham số thường được điều chỉnh tự động (ví dụ: sử dụng tìm kiếm ngẫu nhiên), nhưng việc hiểu rõ chức năng của từng siêu tham số và cách chúng tương tác với các siêu tham số khác là rất hữu ích: bằng cách này, bạn có thể thu hẹp phạm vi tìm kiếm xuống một mức nhỏ hơn nhiều.

không gian.

Đặc điểm tương đồng Một ký

thuật khác để giải quyết các vấn đề phi tuyến tính là thêm các đặc điểm được tính toán bằng cách sử dụng hàm tương đồng, đo lường mức độ mỗi trường hợp giống với một mốc cụ thể, như chúng ta đã làm trong [Chương 2](#) khi thêm các đặc điểm tương đồng địa lý. Ví dụ, hãy lấy tập dữ liệu 1D đã thảo luận trước đó và thêm hai mốc vào đó tại $x = -2$ và $x = 1$ (xem biểu đồ bên trái trong [Hình 5-8](#)). Tiếp theo, hãy định nghĩa hàm tương đồng là hàm RBF Gaussian với $\gamma = 0,3$. Đây là một hàm hình chuông biến thiên từ 0 (rất xa mốc) đến 1 (tại mốc).

Giờ chúng ta đã sẵn sàng tính toán các đặc trưng mới. Ví dụ, hãy xem xét trường hợp $x = -1$: nó nằm cách mốc thứ nhất một khoảng 1 và cách mốc thứ hai một khoảng 2. Do đó, các đặc trưng mới của nó là $x = \exp(-0.3 \times 1^2) = 0.74$ và $x = \exp(-0.3 \times 2^2) = 0.30$. Biểu đồ bên phải trong [Hình 5-8](#) hiển thị tập dữ liệu đã được biến đổi 3 (loại bỏ các đặc trưng ban đầu). Như [hình 5-8](#) có thể thấy, giờ đây nó có thể phân tách tuyến tính.



Hình 5-8. Các đặc điểm tương đồng sử dụng RBF Gaussian

Bạn có thể thắc mắc làm thế nào để chọn các điểm mốc. Cách tiếp cận đơn giản nhất là tạo một điểm mốc tại vị trí của mỗi trường hợp trong tập dữ liệu. Làm như vậy sẽ tạo ra nhiều chiều và do đó làm tăng khả năng tập huấn luyện được chuyển đổi sẽ phân tách tuyến tính. Nhược điểm là một tập huấn luyện với m trường hợp và n

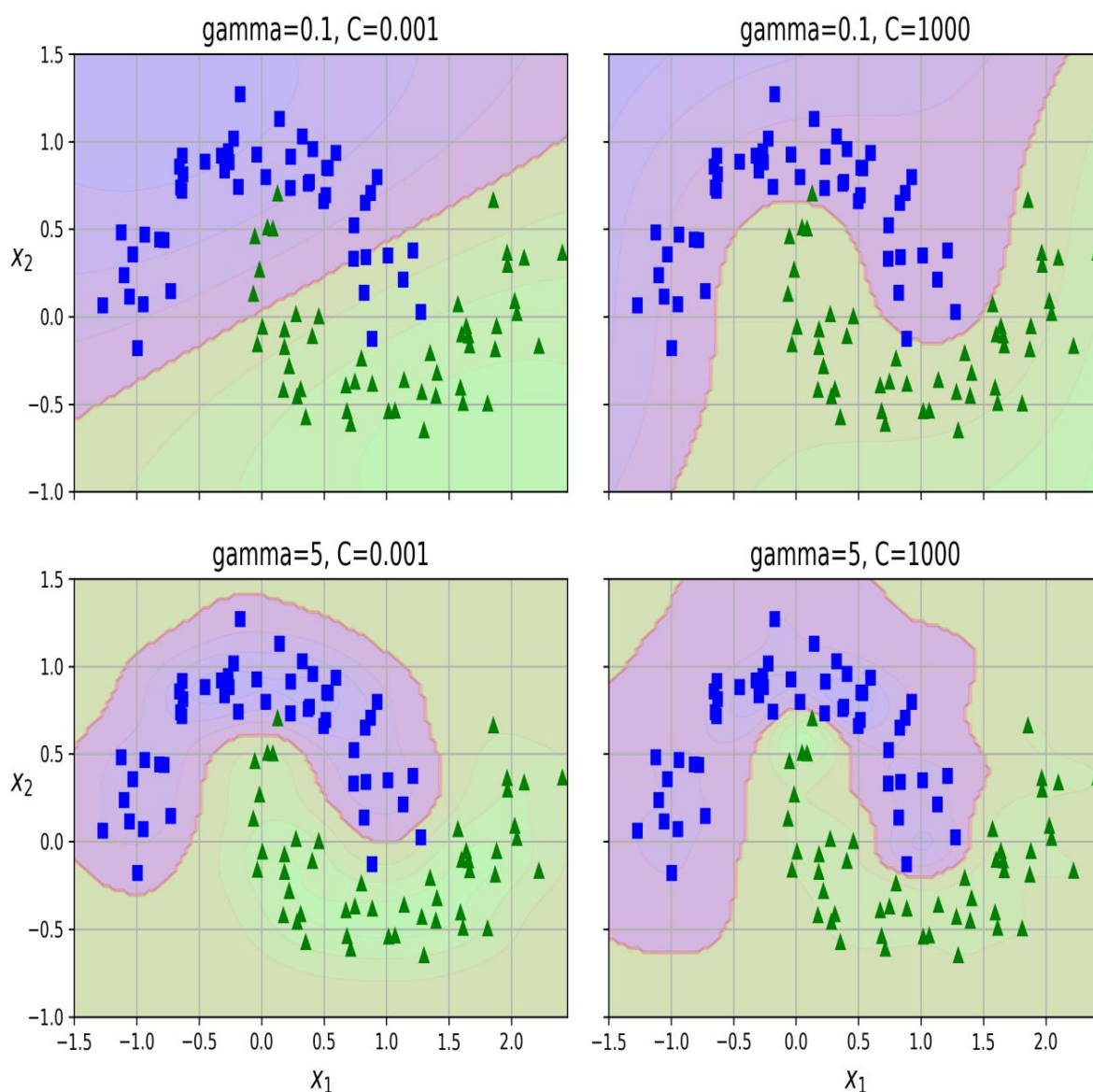
Các đặc trưng được chuyển đổi thành một tập huấn luyện với m trường hợp và m đặc trưng (giả sử bạn loại bỏ các đặc trưng ban đầu). Nếu tập huấn luyện của bạn rất lớn, bạn sẽ có một số lượng đặc trưng lớn tương ứng.

Hạt nhân RBF Gaussian

Cũng giống như phương pháp đặc trưng đa thức, phương pháp đặc trưng tương đồng có thể hữu ích với bất kỳ thuật toán Học máy nào, nhưng việc tính toán tất cả các đặc trưng bổ sung có thể tốn kém về mặt tính toán, đặc biệt là trên các tập dữ liệu huấn luyện lớn. Một lần nữa, thủ thuật kernel lại phát huy tác dụng kỳ diệu của SVM, giúp thu được kết quả tương tự như thể bạn đã thêm nhiều đặc trưng tương đồng, nhưng thực tế lại không cần làm vậy. Hãy thử lớp SVC với kernel Gaussian RBF:

```
rbf_kernel_svm_clf = make_pipeline(StandardScaler(),
                                   SVC(kernel="rbf", gamma=5, C=0.001))
rbf_kernel_svm_clf.fit(X, y)
```

Mô hình này được thể hiện ở phía dưới bên trái trong [Hình 5-9](#). Các đồ thị khác hiển thị các mô hình được huấn luyện với các giá trị khác nhau của siêu tham số gamma (γ) và C. Việc tăng gamma làm cho đường cong hình chuông hẹp hơn (xem các đồ thị bên trái trong [Hình 5-8](#)). Kết quả là, phạm vi ảnh hưởng của mỗi trường hợp nhỏ hơn: ranh giới quyết định trở nên không đều hơn, uốn lượn xung quanh các trường hợp riêng lẻ. Ngược lại, giá trị gamma nhỏ làm cho đường cong hình chuông rộng hơn: các trường hợp có phạm vi ảnh hưởng lớn hơn, và ranh giới quyết định trở nên mượt mà hơn. Vì vậy, γ hoạt động như một siêu tham số điều chỉnh: nếu mô hình của bạn bị quá khớp, bạn nên giảm γ ; nếu nó bị thiếu khớp, bạn nên tăng γ (tương tự như siêu tham số C).



Hình 5-9. Bộ phân loại SVM sử dụng hạt nhân RBF

Các loại kernel khác cũng tồn tại nhưng được sử dụng ít hơn nhiều. Một số kernel được chuyên dụng cho các cấu trúc dữ liệu cụ thể. Kernel chuỗi đôi khi được sử dụng khi phân loại tài liệu văn bản hoặc trình tự DNA (ví dụ: sử dụng kernel chuỗi con hoặc kernel dựa trên khoảng cách Levenshtein).

MẸO

Với rất nhiều kernel để lựa chọn, làm sao bạn có thể quyết định nên sử dụng kernel nào? Theo nguyên tắc chung, bạn nên thử kernel tuyến tính trước. Lớp LinearSVC nhanh hơn nhiều so với SVC(kernel="linear"), đặc biệt nếu tập dữ liệu huấn luyện rất lớn. Nếu tập dữ liệu không quá lớn, bạn cũng nên thử SVM sử dụng kernel, bắt đầu với kernel Gaussian RBF; nó thường hoạt động rất tốt. Sau đó, nếu bạn có thời gian và sức mạnh tính toán, bạn có thể thử nghiệm với một vài kernel khác, sử dụng tìm kiếm siêu tham số. Nếu có các kernel chuyên dụng cho cấu trúc dữ liệu của tập dữ liệu huấn luyện của bạn, hãy chắc chắn thử chúng.

Các lớp SVM và độ phức tạp tính toán

Lớp LinearSVC dựa trên thư viện liblinear, thư viện này triển khai một thuật toán được tối ưu hóa. Đối với SVM tuyến tính, nó không hỗ trợ thủ thuật kernel, nhưng hiệu suất của nó tăng gần như tuyến tính với số lượng mẫu huấn luyện.

số lượng đặc trưng. Độ phức tạp thời gian huấn luyện của nó xấp xỉ $O(m \times n)$. Thuật toán sẽ mất nhiều thời gian hơn nếu bạn yêu cầu độ chính xác rất cao. Điều này được kiểm soát bởi siêu tham số dung sai ϵ (được gọi là tol trong Scikit-Learn). Trong hầu hết các trường hợp, cho các tác vụ phân loại, dung sai mặc định là phù hợp.

Lớp SVC dựa trên thư viện libsvm, thư viện này triển khai **một thuật toán**. Điều này hỗ trợ thủ thuật kernel. Độ phức tạp thời gian huấn luyện thường nằm giữa $O(m \times n)$ và $O(m^2 \times n)$. Thật không may,²điều này có nghĩa³là nó trở nên cực kỳ chậm khi số lượng mẫu huấn luyện lớn (ví dụ: hàng trăm nghìn mẫu). Thuật toán này phù hợp nhất cho các tập huấn luyện phi tuyến tính có kích thước nhỏ hoặc trung bình. Nó có khả năng mở rộng tốt với số lượng đặc trưng, đặc biệt là với các đặc trưng thưa (tức là khi mỗi mẫu có ít đặc trưng khác 0). Trong trường hợp này, thuật toán có khả năng mở rộng xấp xỉ với số lượng đặc trưng khác 0 trung bình trên mỗi mẫu. **Bảng 5-1** so sánh các lớp phân loại SVM của Scikit-Learn.

Cuối cùng, lớp SGDClassifier cũng thực hiện phân loại biên độ lớn theo mặc định, và các siêu tham số của nó có thể được điều chỉnh để tạo ra kết quả tương tự như SVM tuyến tính, đặc biệt là các siêu tham số điều chỉnh (alpha và penalty) và learning_rate. Để huấn luyện, nó sử dụng thuật toán Stochastic Gradient Descent (xem **Chương 4**), cho phép học tăng dần và sử dụng ít bộ nhớ, vì vậy bạn có thể sử dụng nó để huấn luyện mô hình trên tập dữ liệu lớn không vừa trong RAM (tức là ngoài bộ nhớ). Hơn nữa, nó có khả năng mở rộng rất tốt, vì độ phức tạp tính toán của nó là $O(m \times n)$.

T

Một

bl

e

5-

1.

C

o

m

p

Một

ri

S

o

N

của

S

ci

ki

t-

L

e

Một

r

N

cl

Một

ss

es

cho

r

S

V

M

cl

Một

ss

ifi

C

Tại

io

N

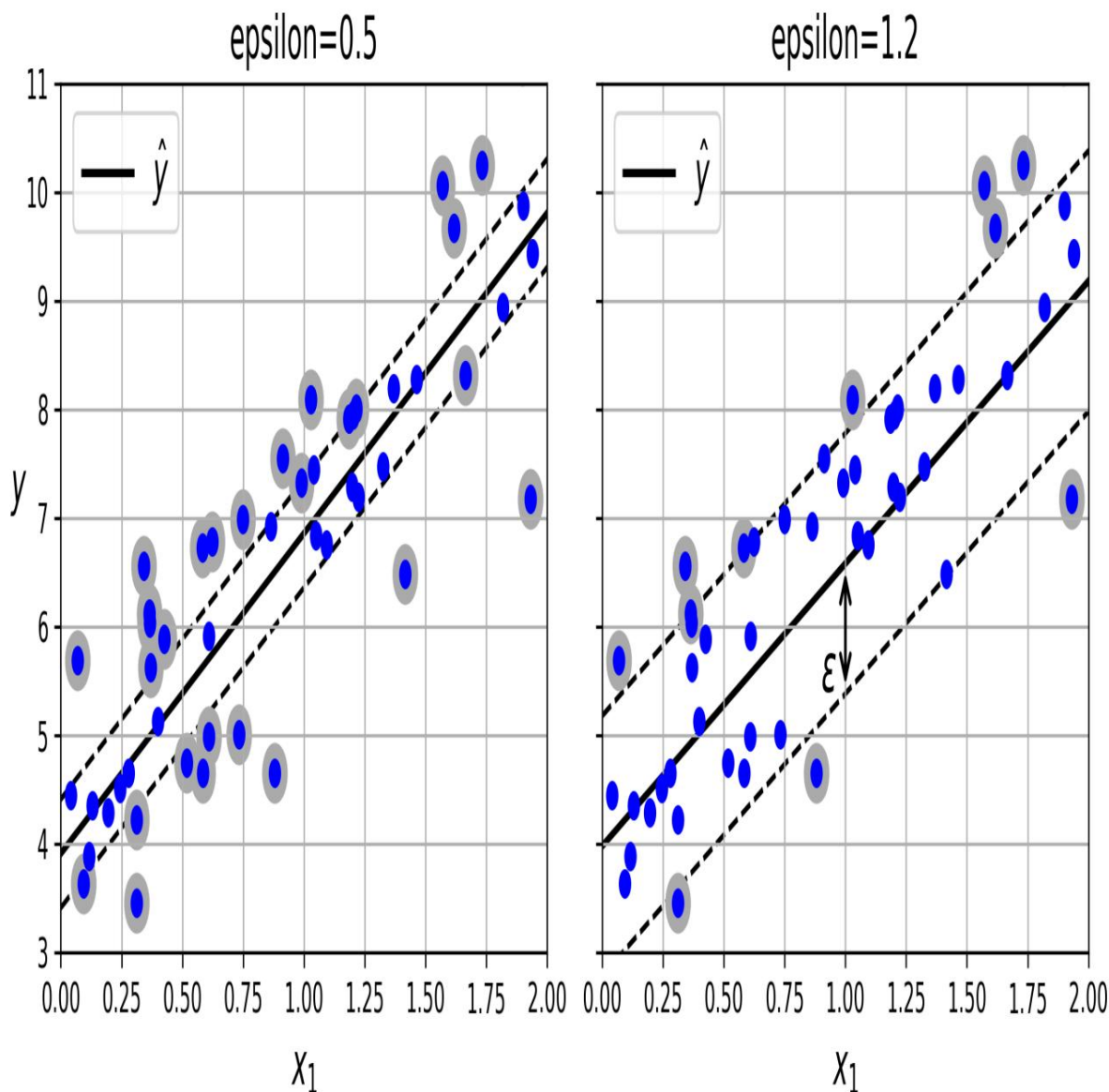
Lớp học				
Độ phức tạp về thời gian Hỗ trợ ngoài bộ nhớ Khả năng mở rộng cần thiết Thủ thuật nhân				
LinearSVC	$O(m \times n)$	không	Đúng	không
SVC	$O(m^2 \times n)$ đến $O(m^3 \times n)$	Không	Đúng	Đúng
SGDClassifier	$O(m \times n)$	Đúng	Đúng	không

Bây giờ chúng ta hãy xem thuật toán SVM có thể được sử dụng như thế nào cho hồi quy tuyến tính và phi tuyến tính.

Hồi quy SVM

Để sử dụng SVM cho hồi quy thay vì phân loại, mẹo là điều chỉnh hàm mục tiêu: thay vì cố gắng khớp...

Để tìm được con đường lớn nhất có thể giữa hai lớp trong khi hạn chế tối đa các vi phạm về lề, SVM Regression cố gắng phù hợp với càng nhiều càng tốt. càng nhiều trường hợp càng tốt trên đường phố trong khi hạn chế vi phạm lề đường (tức là các trường hợp ngoài đường phố). Chiều rộng của Đường phố được điều khiển bởi một siêu tham số, ϵ . Hình 5-10 hiển thị hai mô hình hồi quy SVM tuyến tính được huấn luyện trên một số dữ liệu tuyến tính, một dữ liệu có biên độ nhỏ ($\epsilon = 0,5$) và dữ liệu kia có biên độ lớn hơn ($\epsilon = 1,2$).



Hình 5-10. Hồi quy SVM

Việc giảm ϵ việc này làm tăng số lượng vector hỗ trợ, giúp chuẩn hóa mô hình. Hơn nữa, nếu bạn thêm nhiều hơn nữa, số lượng mẫu huấn luyện trong phạm vi cho phép sẽ không ảnh hưởng đến dự đoán của mô hình; do đó, mô hình được gọi là không nhạy ϵ -cảm.

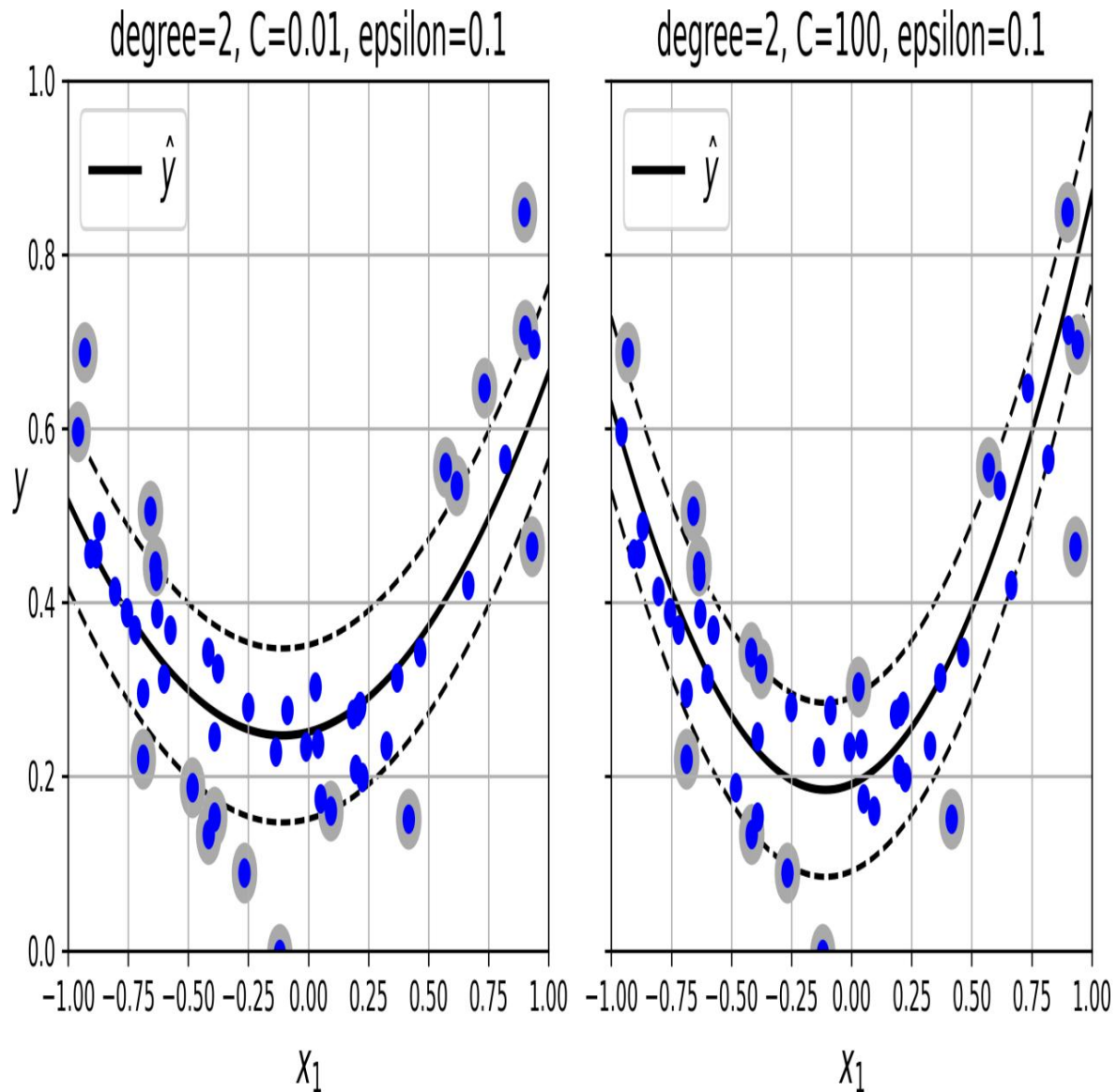
Bạn có thể sử dụng lớp `LinearSVR` của `Scikit-Learn` để thực hiện hồi quy SVM tuyến tính. Đoạn mã sau đây tạo ra kết quả như sau: Mô hình được thể hiện ở bên trái trong Hình 5-10:

```
from sklearn.svm import LinearSVR

X, y = [...] # một tập dữ liệu tuyến tính
svm_reg = make_pipeline(StandardScaler(),
                        LinearSVR(epsilon=0.5, random_state=42))

svm_reg.fit(X, y)
```

Để giải quyết các bài toán hồi quy phi tuyến tính, bạn có thể sử dụng mô hình SVM có nhân (kernelized SVM). Hình 5-11 minh họa hồi quy SVM trên tập dữ liệu huấn luyện bậc hai ngẫu nhiên, sử dụng nhân đa thức bậc hai. Có một số điều chỉnh (regularization) trong biểu đồ bên trái (tức là giá trị C nhỏ), và ít hơn nhiều trong biểu đồ bên phải (tức là giá trị C lớn).



Hình 5-11. Hồi quy SVM sử dụng hạt nhân đa thức bậc hai.

Đoạn mã sau sử dụng lớp SVR của Scikit-Learn (hỗ trợ thủ thuật kernel) để tạo ra mô hình được thể hiện ở bên trái trong Hình 5-11:

```
from sklearn.svm import SVR

X, y = [...] # một tập dữ liệu bậc hai
svm_poly_reg = make_pipeline(StandardScaler(),
```

```
SVR(kernel="poly", degree=2, C=0.01, epsilon=0.1))\n\nsvm_poly_reg.fit(X, y)
```

Lớp SVR là mô hình hồi quy tương đương với lớp SVC, và lớp LinearSVR là mô hình hồi quy tương đương với lớp LinearSVC. Lớp LinearSVR có khả năng mở rộng tuyến tính với kích thước của tập dữ liệu huấn luyện (chỉ giống như lớp LinearSVC), trong khi lớp SVR trở nên quá chậm khi tập dữ liệu huấn luyện tăng lên rất lớn (chỉ giống như lớp SVC).

GHI CHÚ

SVM cũng có thể được sử dụng để phát hiện sự mới lạ, như chúng ta sẽ thấy trong [Chương 9](#).

Phần còn lại của chương này giải thích cách SVM đưa ra dự đoán và cách các thuật toán huấn luyện của chúng hoạt động, bắt đầu từ... với bộ phân loại SVM tuyến tính. Nếu bạn mới bắt đầu với Học máy, bạn có thể bỏ qua phần này và tiếp tục... Hãy chuyển thẳng đến phần bài tập ở cuối chương này, và quay lại sau khi bạn muốn tìm hiểu sâu hơn. Hiểu biết về SVM.

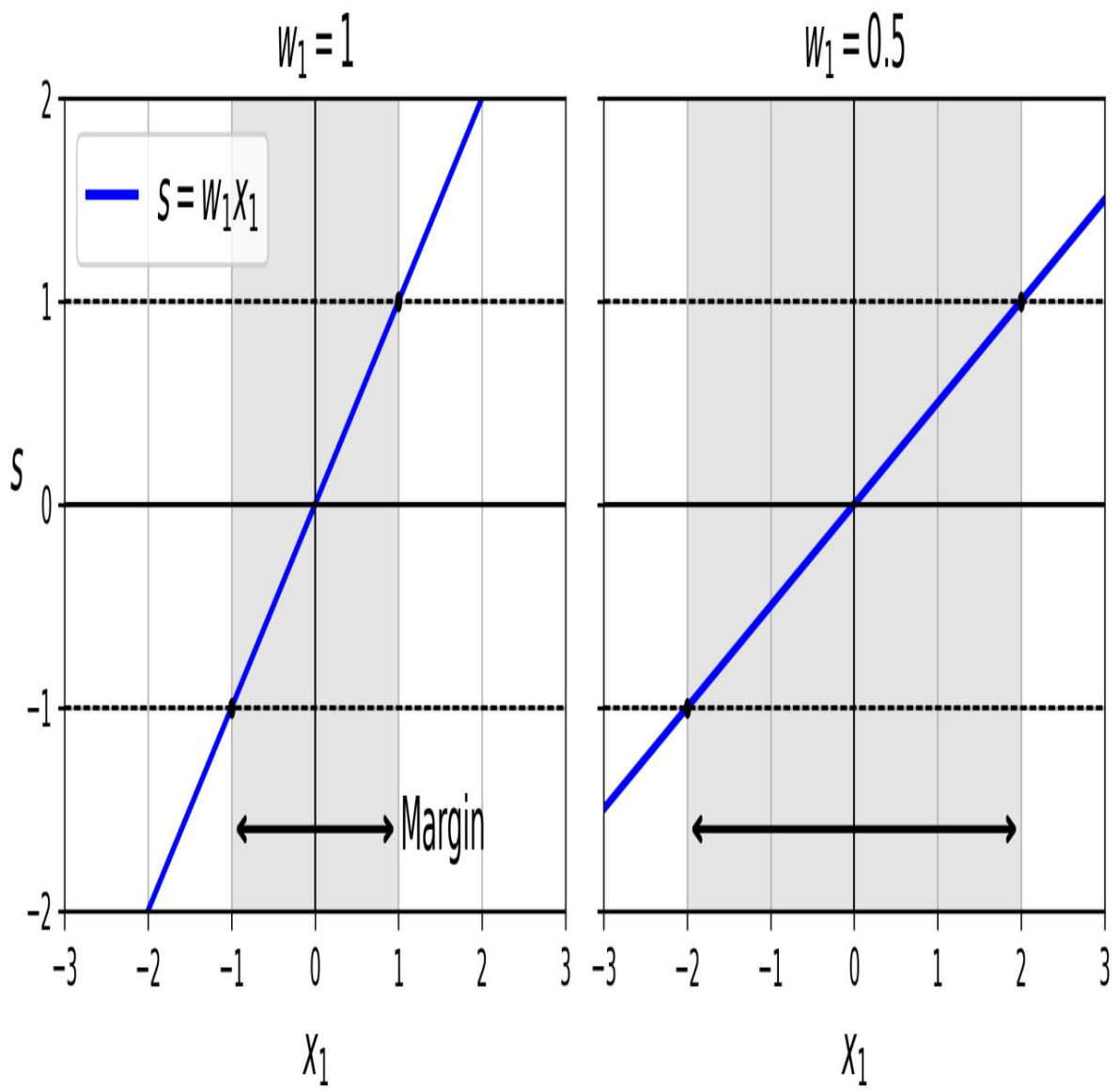
Bên trong bộ phân loại SVM tuyến tính

Mô hình phân loại SVM tuyến tính dự đoán lớp của một trường hợp mới x bằng cách tính toán hàm quyết định θ trước tiên. $x = \theta_0 + \theta_1 x_1$, trong đó θ_0 là đặc trưng độ lệch (luôn bằng 1). Nếu kết quả dương, thì dự đoán y là lớp tích cực (1), nếu không thì là lớp tiêu cực (0). Điều này hoàn toàn giống như Hồi quy Logistic (được thảo luận trong [Chương 4](#)).

GHI CHÚ

Cho đến nay, tôi vẫn sử dụng quy ước đặt tất cả các tham số của mô hình vào một vectơ θ , bao gồm cả số hạng độ lệch θ_0 và đặc trưng đầu vào. các trọng số θ . Điều này đòi hỏi phải thêm đầu vào độ lệch $x = 1$ vào tất cả các trường hợp. Một quy ước rất phổ biến khác là tách riêng số hạng độ lệch b đến θ (bằng θ_0), và vectơ trọng số đặc trưng w (chứa θ đến θ). Trong trường hợp này, không cần thêm đặc trưng độ lệch nào vào đặc trưng đầu vào. các vectơ, và hàm quyết định của SVM tuyến tính bằng $w \cdot x + b = w \cdot x + 1 \cdot \theta_0 + b$. Tôi sẽ sử dụng quy ước này trong suốt phần còn lại của cuốn sách này.

Vì vậy, việc dự đoán bằng bộ phân loại SVM tuyến tính khá đơn giản. Còn việc huấn luyện thì sao? Vâng, nó đòi hỏi... Tìm vectơ trọng số w và số hạng độ lệch b sao cho "con đường" (tức là lề đường) rộng nhất có thể trong khi hạn chế số lần vi phạm lề. Chúng ta hãy bắt đầu với chiều rộng của con đường: để làm cho nó rộng hơn, chúng ta cần phải làm gì? w nhỏ hơn. Điều này có thể dễ hình dung hơn trong không gian 2D, như thể hiện trong [Hình 5-12](#). Hãy xác định ranh giới của con phố là các điểm mà hàm quyết định bằng -1 hoặc +1. Trên đồ thị bên trái, trọng số w bằng 1, vì vậy các điểm tại 1 Trong đó $w \cdot x - 1$ hoặc $+1$ là $x = -1$ và $+1$: do đó kích thước lề là 2. Trên biểu đồ bên phải, trọng lượng là 0,5, vì vậy Các điểm mà tại đó $w \cdot x = -1$ hoặc $+1$ là $x = -2$ và $+2$: kích thước lề là 4. Vì vậy, chúng ta cần giữ w càng nhỏ càng tốt. Có thể. Lưu ý rằng hệ số độ lệch b không ảnh hưởng đến kích thước của lề: việc điều chỉnh nó chỉ làm dịch chuyển lề xung quanh, mà không ảnh hưởng đến kích thước của nó.



Hình 5-12. Vectơ trọng số nhỏ hơn dẫn đến biên độ lớn hơn.

Chúng ta cũng muốn tránh vi phạm giới hạn, vì vậy chúng ta cần hàm quyết định lớn hơn 1 cho tất cả các trường hợp huấn luyện tích cực (i) và nhỏ hơn -1 cho các trường hợp huấn luyện tiêu cực. Nếu ta định nghĩa $t = -1$ cho các trường hợp tiêu cực (i) (khi $y = 0$) và $t = 1$ cho các trường hợp tích cực (khi $y = 1$), thì ta có thể viết ràng buộc này là $t(w x + b) \geq 1$ cho tất cả các trường hợp.

Do đó, chúng ta có thể biểu diễn mục tiêu của bộ phân loại SVM tuyến tính biên cứng dưới dạng bài toán tối ưu hóa có ràng buộc trong Phương trình 5-1.

Phương trình 5-1. Mục tiêu của bộ phân loại SVM tuyến tính biên cứng

giảm thiểu w, b $\frac{1}{2} \|w\|^2$

với điều kiện $t(i) (w \cdot x(i) + b) \geq 1$ đối với $i = 1, 2, \dots, m$

GHI CHÚ

Chúng ta đang tối thiểu hóa $\frac{1}{2} \mathbf{w}^T \mathbf{w}$, bằng $\frac{1}{2} \|\mathbf{w}\|^2$, chứ không phải tối thiểu hóa $\mathbf{w}^T \mathbf{w}$ (chuẩn của $\mathbf{w}$). Thật vậy, $\frac{1}{2} \|\mathbf{w}\|^2$ có đạo hàm đơn giản² (chính là $\mathbf{w}$), trong khi $\mathbf{w}^T \mathbf{w}$ không khả vi tại $\mathbf{w} = \mathbf{0}$. Các thuật toán tối ưu hóa thường hoạt động tốt hơn nhiều trên các hàm khả vi.

Để đạt được mục tiêu biên độ mềm, chúng ta cần đưa vào một biến bù $\zeta_i \geq 0$ cho mỗi trường hợp⁽¹⁰⁾: ζ_i đo lường mức độ mà trường hợp thứ i được phép vi phạm biên độ. Bây giờ chúng ta có hai mục tiêu mâu thuẫn: làm cho các biến bù càng nhỏ càng tốt để giảm thiểu vi phạm biên độ, và làm cho $\frac{1}{2} \mathbf{w}^T \mathbf{w}$ càng nhỏ càng tốt để tăng biên độ. Đây là lúc siêu tham số C xuất hiện: nó cho phép chúng ta xác định sự đánh đổi giữa hai mục tiêu này. Điều này mang lại cho chúng ta bài toán tối ưu hóa có ràng buộc trong **Phương trình 5-2**.

Phương trình 5-2. Mục tiêu của bộ phân loại SVM tuyến tính biên mềm

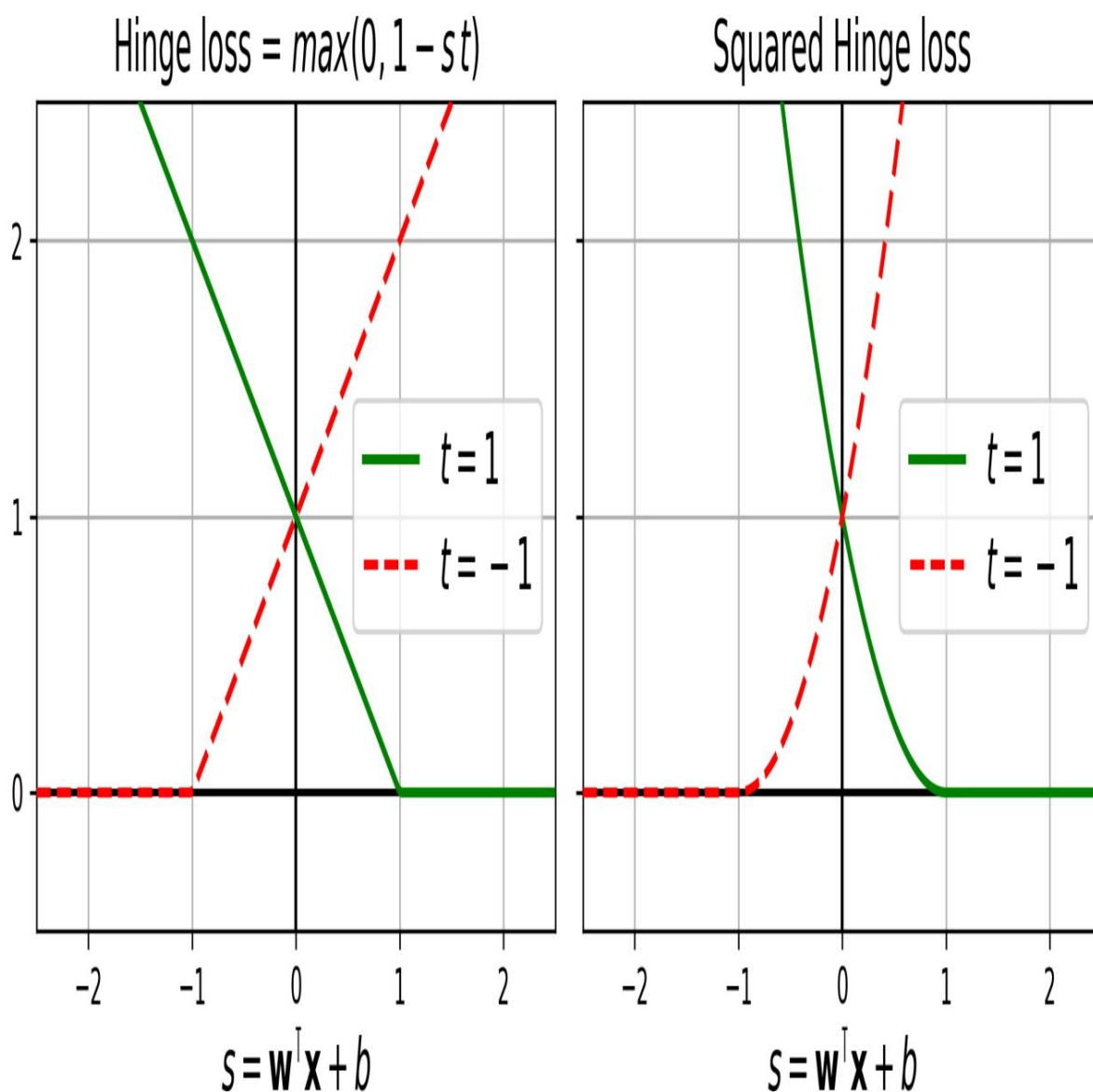
giảm thiểu
 $\mathbf{w}, \mathbf{b}, \zeta$

$$\frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^m \zeta_i$$

$$\text{subject to } t(i) (\mathbf{w}^T \mathbf{x}(i) + b) \geq 1 - \zeta_i \text{ and } \zeta_i \geq 0 \text{ for } i = 1, 2, \dots, m$$

Bài toán biên cứng và biên mềm đều là các bài toán tối ưu hóa bậc hai với các ràng buộc tuyến tính. Những bài toán như vậy được gọi là bài toán Lập trình bậc hai (Quadratic Programming - QP). Có nhiều phần mềm giải toán QP có sẵn trên thị trường bằng cách sử dụng nhiều kỹ thuật khác nhau nằm ngoài phạm vi của cuốn sách này.

Vì vậy, sử dụng bộ giải QP là một cách để huấn luyện SVM. Một cách khác là sử dụng Gradient Descent để tối thiểu hóa tổn thất Hinge hoặc tổn thất Hinge bình phương (xem **Hình 5-13**). Cho một trường hợp $\mathbf{x}$ thuộc lớp tích cực (tức là với $t = 1$), tổn thất bằng 0 nếu đầu ra s của hàm quyết định ($s = \mathbf{w}^T \mathbf{x} + b$) lớn hơn hoặc bằng 1. Điều này xảy ra khi trường hợp đó nằm ngoài đường phẩy và ở phía tích cực. Cho một trường hợp thuộc lớp tiêu cực (tức là với $t = -1$), tổn thất bằng 0 nếu $s \leq -1$. Điều này xảy ra khi trường hợp đó nằm ngoài đường phẩy và ở phía tiêu cực. Trường hợp càng cách xa phía đúng của lề, tổn thất càng cao: nó tăng tuyến tính đối với tổn thất Hinge và tăng theo bậc hai đối với tổn thất Hinge bình phương. Điều này làm cho tổn thất Hinge bình phương nhạy cảm hơn với các giá trị ngoại lai. Tuy nhiên, nếu tập dữ liệu sạch, nó có xu hướng hội tụ nhanh hơn. Theo mặc định, LinearSVC sử dụng hàm mất mát Hinge bình phương, trong khi SGDClassifier sử dụng hàm mất mát Hinge. Cả hai lớp đều cho phép bạn chọn hàm mất mát bằng cách đặt siêu tham số mất mát thành "hinge" hoặc "squared_hinge". Thuật toán tối ưu hóa của lớp SVC tìm ra một giải pháp tương tự như việc tối thiểu hóa hàm mất mát Hinge.



Hình 5-13. Tồn thất bản lề (bên trái) và tồn thất bản lề bình phương (bên phải)

Còn một cách khác nữa để huấn luyện bộ phân loại SVM tuyến tính: giải quyết bài toán đối ngẫu.

Vấn đề kép

Cho một bài toán tối ưu có ràng buộc, được gọi là bài toán gốc, ta có thể biểu diễn một bài toán khác nhưng có liên quan chặt chẽ, được gọi là bài toán đối ngẫu. Lời giải của bài toán đối ngẫu thường cho một cận dưới của lời giải của bài toán gốc, nhưng trong một số điều kiện, nó có thể có cùng lời giải với bài toán gốc.

May mắn thay, bài toán SVM đáp ứng được các điều kiện này, vì vậy bạn có thể chọn giải bài toán gốc hoặc bài toán đối ngẫu; cả hai đều sẽ có cùng một lời giải. **Phương trình 5-3** thể hiện dạng đối ngẫu của mục tiêu SVM tuyến tính.

Nếu bạn muốn biết cách suy ra bài toán đối ngẫu từ bài toán gốc, hãy xem phần tài liệu bổ sung trong sổ tay.

Phương trình 5-3. Dạng kép của mục tiêu SVM tuyến tính

giảm thiểu
$$\sum_{i=1}^m \sum_{j=1}^m \alpha(i) \alpha(j) t(i) t(j) x(i) \cdot x(j) \quad \text{với } \alpha(i) \geq 0 \text{ cho tất cả } i = 1, 2, \dots, m \text{ và } m$$

tính toán w^* và b^* . Sau đó để tối thiểu hóa phương trình này (sử dụng bộ giải QP), hãy sử dụng **Phương trình 5-4** để khi tìm được vectơ tối thiểu hóa bài toán gốc. Trong phương trình này, n biểu thị số lượng vectơ hỗ trợ.

Phương trình 5-4. Từ nghiệm kép đến nghiệm gốc

$$w^* = \sum_{i=1}^m \alpha^*(i) t(i) x(i) \\ b^* = \frac{1}{ns} \sum_{\substack{i=1 \\ \alpha^*(i) > 0}}^m (t(i) \cdot w^* \cdot x(i))$$

Bài toán kép được giải nhanh hơn bài toán gốc khi số lượng mẫu huấn luyện nhỏ hơn số lượng đặc trưng. Quan trọng hơn, bài toán kép cho phép sử dụng thủ thuật kernel, trong khi bài toán gốc thì không. Vậy rốt cuộc thủ thuật kernel này là gì?

SVM nhân hóa

Giả sử bạn muốn áp dụng phép biến đổi đa thức bậc hai cho một tập dữ liệu huấn luyện hai chiều (chẳng hạn như tập dữ liệu huấn luyện về mặt trăng), sau đó huấn luyện một bộ phân loại SVM tuyến tính trên tập dữ liệu huấn luyện đã được biến đổi. **Phương trình 5-5** thể hiện hàm ánh xạ đa thức bậc hai mà bạn muốn ứng tuyến.

Phương trình 5-5. Ánh xạ đa thức bậc hai

$$\varphi(x) = \begin{pmatrix} x_1 \\ \sqrt{2} x_1 x_2 \\ x_2 \end{pmatrix}$$

Lưu ý rằng vectơ được biến đổi là vectơ 3D thay vì 2D. Bây giờ chúng ta hãy xem điều gì xảy ra với một vài vectơ 2D, a và b , nếu ta áp dụng phép ánh xạ đa thức bậc hai này và sau đó tính tích vô hướng⁶ của các vectơ được biến đổi (Xem **Phương trình 5-6**).

Phương trình 5-6. Thủ thuật hạt nhân cho ánh xạ đa thức bậc hai.

$$\begin{aligned} \varphi(a) \cdot \varphi(b) &= \begin{pmatrix} a_1 \\ \sqrt{2} a_1 a_2 \\ a_2 \end{pmatrix} \cdot \begin{pmatrix} b_1 \\ \sqrt{2} b_1 b_2 \\ b_2 \end{pmatrix} = a_1 b_1 + 2 a_1 a_2 b_1 b_2 + a_2 b_2 \\ &= (a_1 b_1 + a_2 b_2)^2 = \begin{pmatrix} a_1 \\ a_2 \end{pmatrix} \cdot \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} = (a \cdot b)^2 \end{aligned}$$

Thật bất ngờ phải không? Tích vô hướng của các vectơ đã biến đổi bằng bình phương tích vô hướng của các vectơ ban đầu. **vectơ:** $\varphi(a) \cdot \varphi(b) = (a \cdot b)^2$.

Đây là điểm mấu chốt: nếu bạn áp dụng phép biến đổi cho tất cả các trường hợp huấn luyện, thì vấn đề đối ngẫu (xem (j)) **Phương trình 5-3** sẽ chứa tích vô hướng $x(i) \cdot x(j)$ thay vì $\varphi(x(i)) \cdot \varphi(x(j))$. Vì thế, được định nghĩa trong **Phương trình 5-5**, sau đó bạn có thể thay thế tích vô hướng của các vectơ biến đổi này đơn giản bằng $x(i) \cdot x(j)$. Bạn không cần phải biến đổi các trường hợp huấn luyện; chỉ cần thay thế tích vô hướng bằng bình phương của nó trong **Phương trình 5-3**.

Kết quả sẽ hoàn toàn giống như khi bạn bỏ công sức biến đổi tập dữ liệu huấn luyện rồi áp dụng thuật toán SVM tuyến tính, nhưng thủ thuật này giúp toàn bộ quá trình tiết kiệm chi phí tính toán hơn nhiều.

Hàm $K(a, b) = (a \cdot b)^2$ là một hạt nhân đa thức bậc hai. Trong Học máy, hạt nhân là một hàm có khả năng tính tích vô hướng (a) tính toán (hoặc thậm chí biết về) các hạt nhân biến đổi. $\varphi(b)$, chỉ dựa trên các vectơ gốc a và b, mà không cần phải . Phương trình 5-7 liệt kê một số phương pháp được sử dụng phổ biến nhất

Phương trình 5-7. Các hạt nhân chung

Tuyến tính: $K(a, b) = a \cdot b$

Đa thức: $K(a, b) = (y a \cdot b + r)^d$

RBF Gaussian: $K(a, b) = \exp(-\gamma \|a - b\|^2)$

Hàm sigmoid: $K(a, b) = \tanh(y a \cdot b + r)$

ĐỊNH LÝ MERCER

Theo định lý Mercer, nếu một hàm $K(a, b)$ thỏa mãn một vài điều kiện toán học được gọi là điều kiện Mercer (ví dụ: K phải liên tục và đối xứng theo các đối số của nó sao cho $K(a, b) = K(b, a)$, v.v.), thì tồn tại một hàm ánh xạ a và b vào một không gian khác (có thể có chiều cao hơn nhiều) sao cho $K(a, b) = (a \cdot b)$. Bạn có thể sử dụng K làm hạt nhân vì bạn biết nó tồn tại, ngay cả khi bạn không biết nó là gì. Trong trường hợp hạt nhân RBF Gaussian, có thể chứng minh rằng nó ánh xạ mỗi trường hợp huấn luyện vào một không gian vô hạn chiều, vì vậy thật tốt khi bạn không cần phải thực hiện việc ánh xạ!

Lưu ý rằng một số hàm nhân thường dùng (như hàm nhân sigmoid) không tuân thủ tất cả các điều kiện của Mercer, nhưng nhìn chung chúng hoạt động tốt trong thực tế.

Vẫn còn một vấn đề chưa được giải quyết mà chúng ta cần phải giải quyết. Phương trình 5-4 cho thấy cách chuyển từ nghiệm kép sang nghiệm chính trong trường hợp bộ phân loại SVM tuyến tính. Nhưng nếu bạn áp dụng thủ thuật kernel, bạn sẽ nhận được các phương trình (x). Trên hạn, vì vậy bạn không thể biết trước được điều hướng (xem, theo như bắt đầu từ đầu của hàm dự đoán mà không biết w^{*}? Tin tốt là (n) bạn có thể cắm công thức cho w^{*} từ Phương trình 5-4 vào hàm quyết định cho một trường hợp mới x để nhận được một phương trình chỉ có tích vô hướng giữa các vectơ đầu vào. Điều này cho phép sử dụng thủ thuật kernel (Phương trình 5-8), và bạn

Phương trình 5-8. Đưa ra dự đoán bằng SVM nhân hóa.

$$h(w^*, \hat{b}(\varphi(x(n)))) = w^* \cdot \varphi(x(n)) + \hat{b} = \sum_{i=1}^m \alpha^* (i) t(i) \varphi(x(i)) \cdot \varphi(x(n)) + \hat{b}$$
$$= \sum_{i=1}^m \alpha^* (i) t(i) K(x(i), x(n)) + \hat{b}$$
$$\alpha^* (i) > 0$$

Lưu ý rằng vì α^* chỉ áp dụng cho các vectơ hỗ trợ, nên việc dự đoán liên quan đến việc tính tích vô hướng của vectơ đầu vào mới x với chỉ các vectơ hỗ trợ, chứ không phải tất cả các trường hợp huấn luyện. Tất nhiên, bạn cần sử dụng thủ thuật tương tự để tính số hạng độ lệch $\hat{b}$ (Phương trình 5-9).

Phương trình 5-9. Sử dụng thủ thuật hạt nhân để tính toán số hạng sai lệch.

$$\begin{aligned} \hat{b} &= \frac{1}{n_s} \sum_{i=1}^m (t(i) - w^* \varphi(x(i))) = \frac{1}{n_s} \sum_{i=1}^m (t(i) - (\sum_{j=1}^m \alpha^*(j) t(j)) \varphi(x(i))) \\ &= \frac{1}{n_s} \sum_{i=1}^m \sum_{j=1}^m \alpha^*(j) t(j) K(x(i), x(j)) \end{aligned}$$

Nếu bạn bắt đầu bị đau đầu, điều đó hoàn toàn bình thường: đó là một tác dụng phụ không mong muốn của thủ thuật kernel.

GHI CHÚ

Cũng có thể triển khai các SVM kernelized trực tuyến, có khả năng học tăng dần, như được mô tả trong các bài báo “[Học máy vector hỗ trợ tăng dần và giảm dần](#)” và “[Bộ phân loại kernel nhanh với học trực tuyến và chủ động](#)”.⁷ Các thuật toán SVM dựa trên nhân này được triển khai trong Matlab và C++. Tuy nhiên, đối với các bài toán phi tuyến quy mô lớn, bạn có thể cần nhắc sử dụng Rừng ngẫu nhiên (xem [Chương 7](#)) hoặc mạng nơ-ron (xem [Phần II](#)).

Bài tập

1. Ý tưởng cơ bản đằng sau máy học hỗ trợ vectơ (Support Vector Machines - SVM) là gì?
2. Vectơ hỗ trợ là gì?
3. Tại sao việc điều chỉnh tỷ lệ đầu vào lại quan trọng khi sử dụng SVM?
4. Liệu bộ phân loại SVM có thể đưa ra điểm tin cậy khi phân loại một trường hợp không? Còn về xác suất thì sao?
5. Làm thế nào để lựa chọn giữa LinearSVC, SVC hoặc SGDClassifier?
6. Giả sử bạn đã huấn luyện một bộ phân loại SVM với hạt nhân RBF, nhưng có vẻ như nó không phù hợp với tập dữ liệu huấn luyện. Bạn nên tăng hay giảm γ (gamma)? Còn C thì sao?
7. Mô hình có nghĩa là gì? ϵ - vô cảm?
8. Mục đích của việc sử dụng thủ thuật kernel là gì?
9. Huấn luyện một mô hình LinearSVC trên một tập dữ liệu có thể phân tách tuyến tính. Sau đó, huấn luyện một mô hình SVC và một mô hình SGDClassifier trên cùng tập dữ liệu đó. Tập dữ liệu. Hãy xem liệu bạn có thể khiến chúng tạo ra mô hình gần giống nhau hay không.
10. Huấn luyện một bộ phân loại SVM trên tập dữ liệu Wine, bạn có thể tái tập dữ liệu này bằng cách sử dụng `sklearn.datasets.load_wine()`. Tập dữ liệu này chứa phân tích hóa học của 178 mẫu rượu vang được sản xuất bởi 3 nhà sản xuất khác nhau: mục tiêu là huấn luyện một mô hình phân loại có khả năng dự đoán nhà sản xuất dựa trên phân tích hóa học của rượu vang. Vì bộ phân loại SVM là bộ phân loại nhị phân, bạn sẽ cần sử dụng phương pháp “một đối tất cả” để phân loại cả 3 lớp. Độ chính xác bạn có thể đạt được là bao nhiêu?
11. Huấn luyện và tinh chỉnh mô hình hồi quy SVM trên tập dữ liệu nhà ở California. Bạn có thể sử dụng tập dữ liệu gốc thay vì phiên bản đã được tinh chỉnh mà chúng ta đã sử dụng trong [Chương 2](#). Tập dữ liệu gốc có thể được tải về bằng cách sử dụng `sklearn.datasets.fetch_california_housing()`. Các mục tiêu đại diện cho hàng trăm nghìn đô la. Vì có hơn 20.000 trường hợp, SVM có thể chạy chậm, vì vậy để tinh chỉnh siêu tham số, bạn nên sử dụng ít trường hợp hơn nhiều (ví dụ: 2.000) để kiểm tra nhiều tổ hợp siêu tham số hơn. RMSE của mô hình tốt nhất của bạn là bao nhiêu?

Đáp án cho các bài tập này có sẵn ở cuối tập tài liệu của chương này, tại địa chỉ <https://homl.info/colab3>.

1 Chih-Jen Lin và cộng sự, "Phương pháp giảm tọa độ kép cho SVM tuyến tính quy mô lớn," Kỷ yếu Hội nghị Quốc tế lần thứ 25 về Học máy (2008): 408-415.

2 John Platt, "Tối ưu hóa tối thiểu tuần tự: Thuật toán nhanh để huấn luyện máy vectơ hỗ trợ" (Báo cáo kỹ thuật của Microsoft Research, (Ngày 21 tháng 4 năm 1998), <https://hml.info/smo>.

3 Zeta (ζ) là chữ cái thứ sáu trong bảng chữ cái Hy Lạp.

4. Để tìm hiểu thêm về Lập trình bậc hai, bạn có thể bắt đầu bằng cách đọc cuốn sách *Tối ưu hóa lỗi* của Stephen Boyd và Lieven Vandenberghe. (Nhà xuất bản Đại học Cambridge, 2004) hoặc xem *loạt bài giảng video* của Richard Brown .

5. Hàm mục tiêu là hàm lỗi, và các ràng buộc bất đẳng thức là các hàm lỗi và khả vi liên tục.

6 Như đã giải thích trong *Chương 4*, tích vô hướng của hai vectơ a và b thường được ký hiệu là $a \cdot b$. Tuy nhiên, trong Học máy, các vectơ thường được... được biểu diễn dưới dạng vectơ cột (tức là ma trận một cột), do đó tích vô hướng được thực hiện bằng cách tính $a^T b$. Để nhất quán với phần còn lại của cuốn sách, chúng ta sẽ sử dụng ký hiệu này ở đây, bỏ qua thực tế là về mặt kỹ thuật, điều này dẫn đến một ma trận một ô chứ không phải là một giá trị vô hướng.

7 Gert Cauwenberghs và Tomaso Poggio, "Học máy vectơ hỗ trợ tăng dần và giảm dần", Kỷ yếu Hội nghị quốc tế lần thứ 13 về Hệ thống xử lý thông tin thần kinh (2000): 388-394.

8 Antoine Bordes et al., "Bộ phân loại hạt nhân nhanh với học trực tuyến và chủ động," Tạp chí nghiên cứu máy học 6 (2005): 1579-1619.

Chương 6. Cây quyết định

Với sách điện tử phát hành sớm, bạn sẽ nhận được sách ở dạng sơ khai nhất—nội dung thô và chưa chỉnh sửa của tác giả khi họ viết—nhờ đó bạn có thể tận dụng những công nghệ này rất lâu trước khi các tựa sách được phát hành chính thức.

Đây sẽ là chương thứ 6 của cuốn sách cuối cùng. Kho lưu trữ GitHub là <https://github.com/ageron/handson-ml3>.

Nếu bạn có ý kiến đóng góp về cách chúng tôi có thể cải thiện nội dung và/hoặc ví dụ trong cuốn sách này, hoặc nếu bạn nhận thấy thiếu sót nào trong chương này, vui lòng liên hệ với biên tập viên theo địa chỉ mcronin@oreilly.com.

Cây quyết định là thuật toán học máy đa năng có thể thực hiện cả nhiệm vụ phân loại và hồi quy, thậm chí cả nhiệm vụ đa đầu ra. Chúng là những thuật toán mạnh mẽ, có khả năng phù hợp với các tập dữ liệu phức tạp. Ví dụ, trong [Chương 2](#), bạn đã huấn luyện mô hình `DecisionTreeRegressor` trên tập dữ liệu nhà ở California, và kết quả khớp hoàn hảo (thực tế là quá khớp).

Cây quyết định cũng là thành phần cơ bản của Rừng ngẫu nhiên (xem [Chương 7](#)), một trong những thuật toán Học máy mạnh mẽ nhất hiện nay.

Trong chương này, chúng ta sẽ bắt đầu bằng cách thảo luận về cách huấn luyện, trực quan hóa và đưa ra dự đoán bằng Cây quyết định. Sau đó, chúng ta sẽ xem xét thuật toán huấn luyện CART được sử dụng bởi Scikit-Learn, và chúng ta sẽ thảo luận về cách điều chỉnh cây và sử dụng chúng cho các nhiệm vụ hồi quy. Cuối cùng, chúng ta sẽ thảo luận về một số hạn chế của Cây quyết định.

Huấn luyện và trực quan hóa cây quyết định

Để hiểu về Cây quyết định, chúng ta hãy cùng xây dựng một cây và xem cách nó đưa ra dự đoán.

Đoạn mã sau huấn luyện một `DecisionTreeClassifier` trên tập dữ liệu iris (xem [Chương 4](#)):

```
from sklearn.datasets import load_iris from
sklearn.tree import DecisionTreeClassifier

iris = load_iris(as_frame=True)
X_iris = iris.data[["chiều dài cánh hoa (cm)", "chiều rộng cánh hoa (cm)"]].values
y_iris = iris.target
```

```
tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf.fit(X_iris, y_iris)
```

Bạn có thể trực quan hóa Cây quyết định đã được huấn luyện bằng cách sử dụng phương thức `export_graphviz()` trước tiên để xuất ra một tệp định nghĩa đồ thị có tên là `iris_tree.dot`:

```
from sklearn.tree import export_graphviz

export_graphviz( tree_clf,

                 out_file="iris_tree.dot", feature_names=["
                 chiều dài cánh hoa (cm)", "chiều rộng cánh hoa (cm)"], class_names=iris.target_names,
                 rounded=True, filled=True

                 )
```

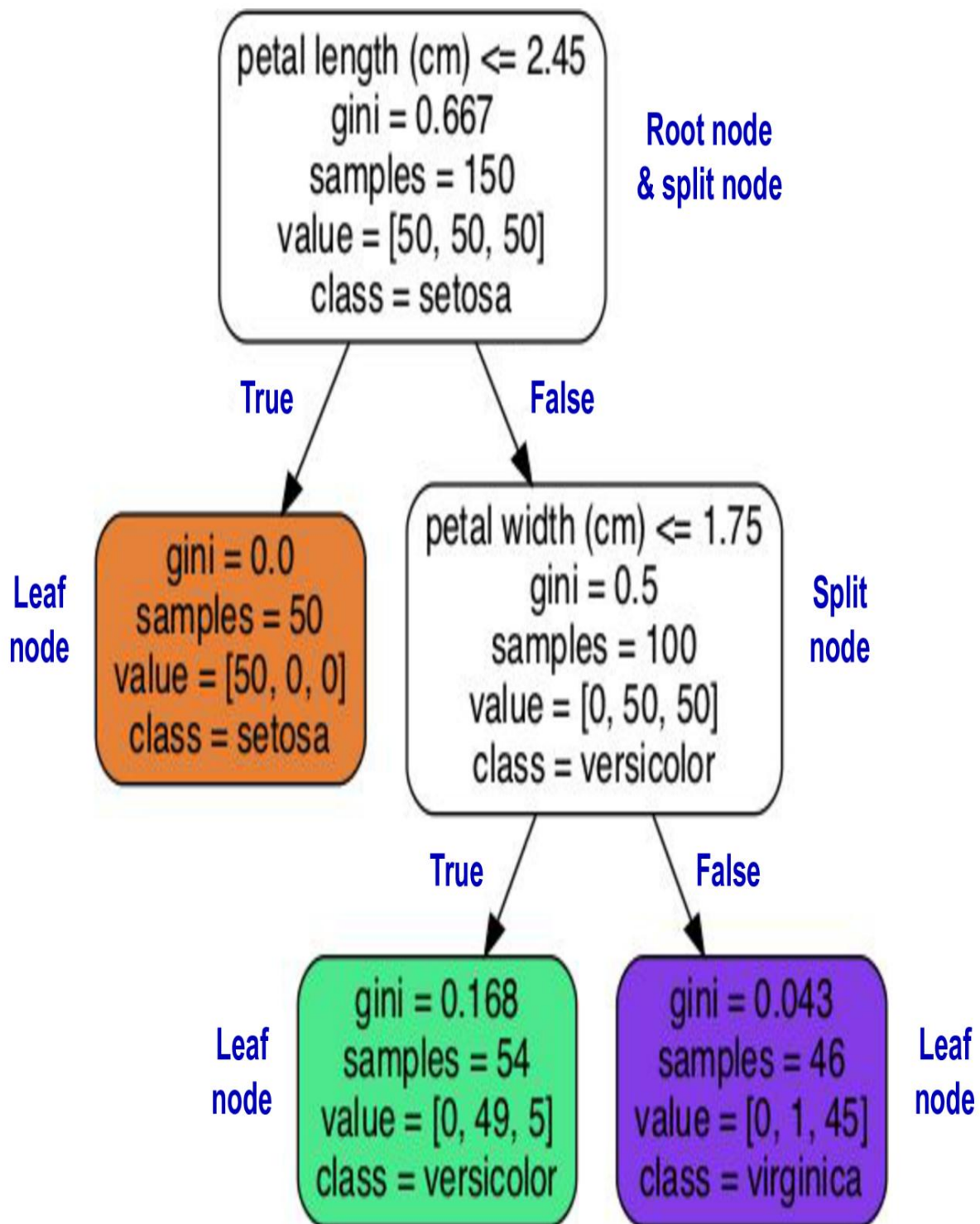
Sau đó, bạn có thể sử dụng `graphviz.Source.from_file()` để tải và hiển thị tệp trong sổ tay Jupyter:

```
from graphviz import Source

Source.from_file("iris_tree.dot")
```

Graphviz Đây là một gói phần mềm trực quan hóa đồ thị mã nguồn mở. Nó cũng bao gồm một công cụ dòng lệnh `dot` để chuyển đổi các tệp `.dot` sang nhiều định dạng khác nhau, chẳng hạn như PDF hoặc PNG.

Cây quyết định đầu tiên của bạn trông giống như **Hình 6-1**.



Hình 6-1. Cây quyết định Iris

Đưa ra dự đoán

Hãy xem cây được biểu diễn trong **Hình 6-1** đưa ra dự đoán như thế nào. Giả sử bạn tìm thấy một cây diên vĩ và muốn phân loại nó dựa trên cánh hoa. Bạn bắt đầu từ nút gốc (độ sâu 0, ở trên cùng): nút này hỏi liệu chiều dài cánh hoa có nhỏ hơn 2,45 cm hay không. Nếu có, bạn di chuyển xuống nút con bên trái của gốc (độ sâu 1, bên trái). Trong trường hợp này, đó là một nút lá (tức là nó không có bất kỳ nút con nào), vì vậy nó không hỏi bất kỳ câu hỏi nào: chỉ cần nhìn vào lớp được dự đoán cho nút đó, và Cây Quyết định dự đoán rằng hoa của bạn là *Iris setosa* (lớp = *setosa*).

Bây giờ giả sử bạn tìm thấy một bông hoa khác, và lần này chiều dài cánh hoa lớn hơn 2,45 cm. Bạn phải di chuyển xuống nút con bên phải của rễ (độ sâu 1, bên phải), đây không phải là nút lá, mà là nút phân nhánh, vì vậy nó đặt ra một câu hỏi khác: chiều rộng cánh hoa có nhỏ hơn 1,75 cm không? Nếu có, thì bông hoa của bạn rất có thể là *Iris versicolor* (độ sâu 2, bên trái). Nếu không, nó có thể là *Iris virginica* (độ sâu 2, bên phải). Thật đơn giản như vậy đấy.

GHI CHÚ

Một trong nhiều ưu điểm của cây quyết định là chúng yêu cầu rất ít bước chuẩn bị dữ liệu. Trên thực tế, chúng không yêu cầu chuẩn hóa hay căn giữa các đặc trưng.

Thuộc tính samples của một nút đếm số lượng mẫu huấn luyện mà nó áp dụng. Ví dụ, có 100 mẫu huấn luyện có chiều dài cánh hoa lớn hơn 2,45 cm (độ sâu 1, bên phải), và trong số 100 mẫu đó, có 54 mẫu có chiều rộng cánh hoa nhỏ hơn 1,75 cm (độ sâu 2, bên trái). Thuộc tính value của một nút cho biết số lượng mẫu huấn luyện của mỗi lớp mà nút này áp dụng: ví dụ, nút dưới cùng bên phải áp dụng cho 0 mẫu *Iris setosa*, 1 mẫu *Iris versicolor* và 45 mẫu *Iris virginica*. Cuối cùng, thuộc tính gini của một nút đo độ không thuần khiết Gini của nó: một nút được gọi là “thuần khiết” ($gini=0$) nếu tất cả các mẫu huấn luyện mà nó áp dụng đều thuộc cùng một lớp. Ví dụ, vì nút bên trái ở độ sâu 1 chỉ áp dụng cho các mẫu huấn luyện *Iris setosa*, nên nó là thuần khiết và độ không thuần khiết Gini của nó là 0. **Phương trình 6-1** cho thấy cách thuật toán huấn luyện tính toán độ không thuần khiết $Gini^{th}$ của nút thứ i . Nút bên trái ở độ sâu 2 có độ tạp chất Gini bằng $1 - (0/54) - (49/54) - (57/54) \approx 0,168$.

Phương trình 6-1. Tạp chất Gini

$$Gi = 1 - \sum_{k=1}^n p_{i,k}^2$$

Trong phương trình này:

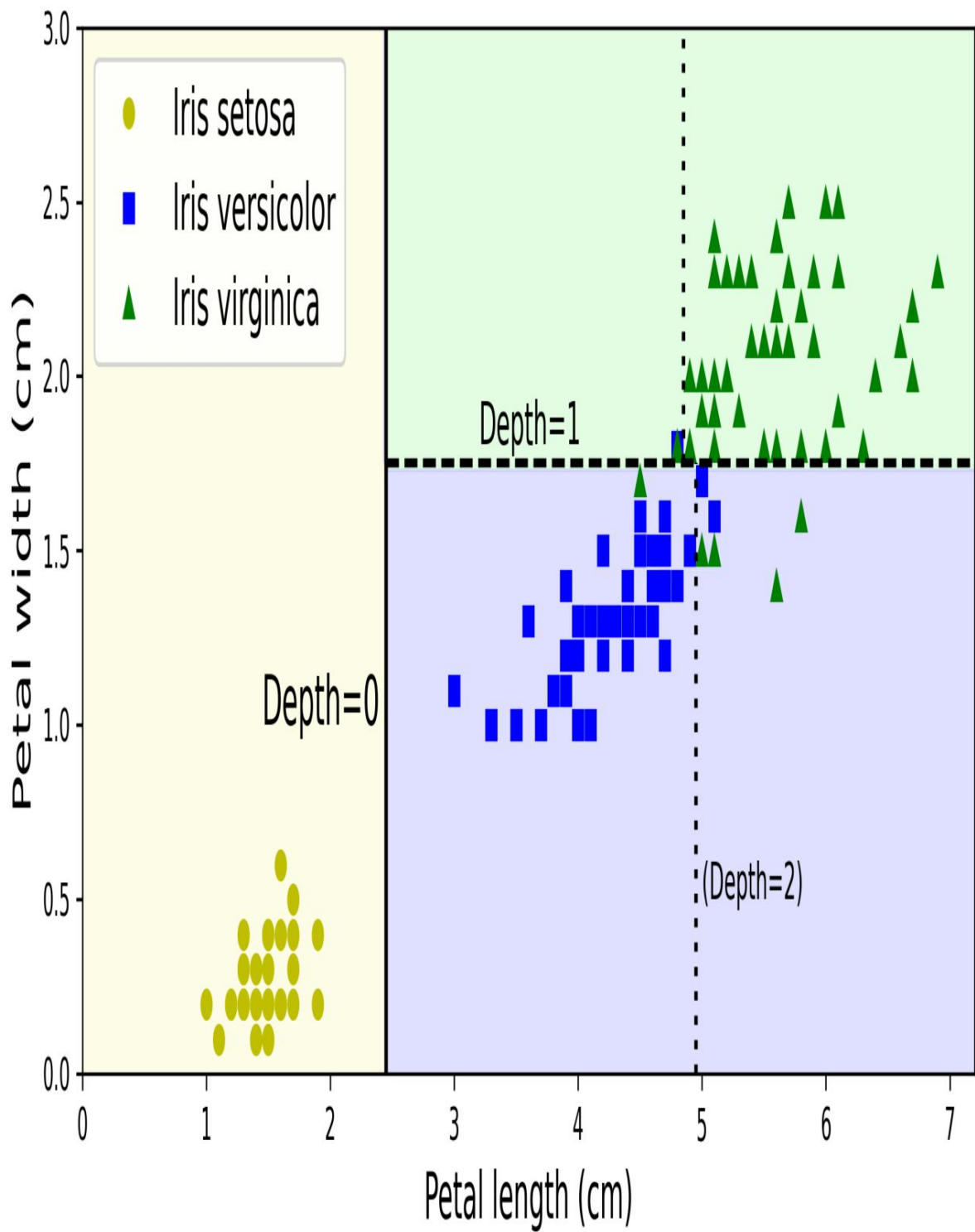
- G là độ tạp chất Gini của nút thứ i .

- p là tỷ lệ các trường hợp thuộc lớp k trong số các trường hợp huấn luyện tại nút i . i, k th

GHI CHÚ

Scikit-Learn sử dụng thuật toán CART, chỉ tạo ra cây nhị phân, nghĩa là cây mà các nút phân nhánh luôn có chính xác hai nút con (tức là câu hỏi chỉ có câu trả lời có/không). Tuy nhiên, các thuật toán khác như ID3 có thể tạo ra Cây Quyết định với các nút có nhiều hơn hai nút con.

Hình 6-2 thể hiện ranh giới quyết định của Cây Quyết định này. Đường thẳng đứng dày biểu thị ranh giới quyết định của nút gốc (độ sâu 0): chiều dài cánh hoa = 2,45 cm. Vì vùng bên trái là thuần chủng (chỉ có Iris setosa), nên không thể chia nhỏ hơn nữa. Tuy nhiên, vùng bên phải là không thuần chủng, vì vậy nút bên phải ở độ sâu 1 chia nó tại chiều rộng cánh hoa = 1,75 cm (được biểu thị bằng đường nét đứt). Vì max_depth được đặt là 2, nên Cây Quyết định dừng lại ngay tại đó. Nếu bạn đặt max_depth là 3, thì hai nút ở độ sâu 2 sẽ thêm mỗi nút một ranh giới quyết định khác (được biểu thị bằng hai đường chấm dọc).



Hình 6-2. Ranh giới quyết định của cây quyết định

GIẢI THÍCH MÔ HÌNH: HỘP TRẮNG SO VỚI HỘP ĐEN

Cây quyết định rất trực quan, và các quyết định của chúng dễ hiểu. Những mô hình như vậy thường được gọi là mô hình hộp trắng. Ngược lại, như chúng ta sẽ thấy, Rừng ngẫu nhiên hoặc mạng nơ-ron thường được coi là mô hình hộp đen. Chúng đưa ra những dự đoán tuyệt vời, và bạn có thể dễ dàng kiểm tra các phép tính mà chúng đã thực hiện để đưa ra những dự đoán này; tuy nhiên, thường rất khó để giải thích một cách đơn giản tại sao các dự đoán được đưa ra. Ví dụ, nếu một mạng nơ-ron nói rằng một người cụ thể xuất hiện trong một bức ảnh, rất khó để biết điều gì đã góp phần vào dự đoán này: liệu mô hình có nhận ra mắt của người đó? Miệng của họ? Mũi của họ? Giày của họ? Hay thậm chí là chiếc ghế sofa mà họ đang ngồi? Ngược lại, Cây quyết định cung cấp các quy tắc phân loại đơn giản, dễ hiểu, thậm chí có thể được áp dụng thủ công nếu cần (ví dụ, để phân loại hoa). Lĩnh vực Học máy có thể giải thích (Interpretable ML) nhằm mục đích tạo ra các hệ thống học máy có thể giải thích các quyết định của chúng theo cách mà con người có thể hiểu được. Điều này rất quan trọng trong nhiều lĩnh vực, ví dụ như để đảm bảo hệ thống không đưa ra các quyết định không công bằng.

MẸO

Cấu trúc cây, bao gồm tất cả thông tin được hiển thị trong **Hình 6-1**, có sẵn thông qua thuộc tính `tree_` của bộ phân loại. Gõ `help(tree_clf.tree_)` để biết chi tiết và xem sổ tay hướng dẫn để biết ví dụ.

Ước tính xác suất của các lớp

Cây quyết định cũng có thể ước tính xác suất một trường hợp thuộc về một lớp k cụ thể. Đầu tiên, nó duyệt cây để tìm nút lá cho trường hợp này, sau đó trả về tỷ lệ các trường hợp huấn luyện của lớp k trong nút này. Ví dụ, giả sử bạn tìm thấy một bông hoa có cánh dài 5 cm và rộng 1,5 cm. Nút lá tương ứng là nút bên trái ở độ sâu 2, vì vậy cây quyết định sẽ đưa ra các xác suất sau: 0% cho Iris setosa (0/54), 90,7% cho Iris versicolor (49/54) và 9,3% cho Iris virginica (5/54).

Và nếu bạn yêu cầu nó dự đoán loại, nó sẽ cho ra kết quả là Iris versicolor (loại 1) vì nó có xác suất cao nhất. Hãy cùng kiểm tra điều này:

```
>>> tree_clf.predict_proba([[5, 1.5]]).round(3) array([[0. , 0.907,
0.093]]) >>> tree_clf.predict([[5, 1.5]])
array([1])
```

Tuyệt vời! Hãy lưu ý rằng xác suất ước tính sẽ giống hệt nhau ở bất kỳ vị trí nào khác trong hình chữ nhật phía dưới bên phải của **Hình 6-2** – ví dụ, nếu cánh hoa dài 6 cm và rộng 1,5 cm.

cm chiều rộng (mặc dù có vẻ hiển nhiên rằng trong trường hợp này, rất có thể đó là loài *Iris virginica*).

Thuật toán huấn luyện CART

Scikit-Learn sử dụng thuật toán Cây Phân loại và Hồi quy (CART) để huấn luyện Cây Quyết định (còn được gọi là cây "phát triển"). Thuật toán hoạt động bằng cách chia tập huấn luyện thành hai tập con bằng cách sử dụng một đặc trưng duy nhất k và một ngưỡng t (ví dụ: "chiều dài cánh hoa $\leq 2,45$ cm"). Làm thế nào nó chọn k và t ? Nó tìm kiếm cặp (k, t) tạo ra các tập con thuần khiết nhất, được trọng số hóa theo kích thước của chúng. **Phương trình 6-2** đưa ra hàm chi phí mà thuật toán cố gắng tối thiểu hóa.

Phương trình 6-2. Hàm chi phí CART cho phân loại

$$J(k, t) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}} \quad \text{Công bằng}$$

$G_{\text{left/right}}$ đo lường độ tạp chất của tập con bên trái/bên phải.

Ở đây $\{m_{\text{left/right}}\}$ là số lượng trường hợp trong tập con bên trái/bên phải.

Sau khi thuật toán CART chia thành công tập dữ liệu huấn luyện thành hai phần, nó sẽ chia các tập con bằng cùng một logic, rồi chia các tập con nhỏ hơn nữa, và cứ thế tiếp tục một cách đệ quy. Nó dừng đệ quy khi đạt đến độ sâu tối đa (được xác định bởi siêu tham số `max_depth`), hoặc nếu nó không thể tìm thấy một cách chia nào làm giảm độ không thuần khiết. Một vài siêu tham số khác (sẽ được mô tả sau) kiểm soát các điều kiện dừng bổ sung: `min_samples_split`, `min_samples_leaf`, `min_weight_fraction_leaf` và `max_leaf_nodes`.

CẢNH BÁO

Như bạn thấy, thuật toán CART là một thuật toán tham lam: nó tìm kiếm một cách tham lam sự phân chia tối ưu ở cấp độ cao nhất, sau đó lặp lại quá trình này ở mỗi cấp độ tiếp theo. Nó không kiểm tra xem sự phân chia đó có dẫn đến mức độ tạp chất thấp nhất có thể ở nhiều cấp độ thấp hơn hay không. Một thuật toán tham lam thường tạo ra một giải pháp khá tốt nhưng không đảm bảo là tối ưu.

Thật không may, việc tìm cây tối ưu được biết đến là một bài toán NP-Complete. Nó đòi hỏi ¹thời gian $O(\exp(m))$, khiến bài toán trở nên bất khả thi ngay cả với các tập dữ liệu huấn luyện nhỏ. Đó là lý do tại sao chúng ta phải chấp nhận một giải pháp "khá tốt" khi huấn luyện cây quyết định.

Độ phức tạp tính toán

Việc đưa ra dự đoán yêu cầu phải duyệt qua Cây Quyết định từ gốc đến lá. Cây Quyết định thường được cân bằng gần đúng, vì vậy việc duyệt qua Cây Quyết định yêu cầu phải đi qua khoảng $O(\log(m))$ nút, trong đó $\log(m)$ là logarit nhị phân của m , bằng $\log(m) / \log(2)$. Vì mỗi nút chỉ yêu cầu kiểm tra giá trị của một đặc trưng, nên độ phức tạp dự đoán tổng thể là $O(\log(m))$, không phụ thuộc vào số lượng đặc trưng.

2

Vì vậy, quá trình dự đoán diễn ra rất nhanh, ngay cả khi xử lý các tập dữ liệu huấn luyện lớn.

Thuật toán huấn luyện so sánh tất cả các đặc trưng (hoặc ít hơn nếu `max_features` được thiết lập) trên tất cả các mẫu tại mỗi nút. Việc so sánh tất cả các đặc trưng trên tất cả các mẫu tại mỗi nút dẫn đến độ phức tạp huấn luyện là $O(n \times m \log(m))$.

2

Gini: Tạp chất hay Entropy?

Theo mặc định, lớp `DecisionTreeClassifier` sử dụng thước đo độ không thuần khiết Gini, nhưng bạn có thể chọn thước đo độ không thuần khiết entropy thay thế bằng cách đặt siêu tham số tiêu chí thành "entropy". Khái niệm entropy bắt nguồn từ nhiệt động lực học như một thước đo sự rối loạn phân tử: entropy tiến đến 0 khi các phân tử đứng yên và được sắp xếp tốt. Entropy sau đó lan rộng sang nhiều lĩnh vực khác nhau, bao gồm cả lý thuyết thông tin của Shannon, nơi nó đo lường nội dung thông tin trung bình của một thông điệp, như chúng ta đã thấy trong [Chương 4](#). Entropy bằng 0 khi tất cả các thông điệp đều giống hệt nhau. Trong Học máy, entropy thường được sử dụng như một thước đo độ không thuần khiết: entropy của một tập hợp bằng 0 khi nó chỉ chứa các trường hợp của một lớp duy nhất. [Phương trình 6-3](#) cho thấy định nghĩa entropy của nút thứ i . Ví dụ, nút bên trái độ sâu 2 trong [Hình 6-1](#) có entropy bằng $-(49/54) \log_2(49/54) - (5/54) \log_2(5/54) \approx 0,445$.

th

2

2

Phương trình 6-3. Entropy

$$H_i = - \sum_{k=1}^n p_{i,k} \log_2(p_{i,k})$$

Vậy, bạn nên sử dụng độ không thuần khiết Gini hay entropy? Sự thật là, hầu hết thời gian sự khác biệt không lớn: chúng tạo ra các cây tương tự nhau. Độ không thuần khiết Gini tính toán nhanh hơn một chút, vì vậy nó là một lựa chọn mặc định tốt. Tuy nhiên, khi chúng khác nhau, độ không thuần khiết Gini có xu hướng cô lập lớp phổ biến nhất vào nhánh riêng của nó trong cây, trong khi entropy có xu hướng tạo ra các cây cân bằng hơn một chút.

2

Siêu tham số điều chỉnh

Cây quyết định đưa ra rất ít giả định về dữ liệu huấn luyện (ngược lại với các mô hình tuyến tính, giả định rằng dữ liệu là tuyến tính chẳng hạn). Nếu không bị ràng buộc, cấu trúc cây sẽ tự thích nghi với dữ liệu huấn luyện, khớp rất sát với dữ liệu - thậm chí rất có thể dẫn đến hiện tượng quá khớp (overfitting). Mô hình như vậy thường được gọi là mô hình phi tham số, không phải vì nó không có tham số (nó thường có rất nhiều) mà vì số lượng tham số không được xác định trước khi huấn luyện, do đó cấu trúc mô hình có thể tự do bám sát dữ liệu. Ngược lại, một mô hình tham số, chẳng hạn như mô hình tuyến tính, có số lượng tham số được xác định trước, do đó mức độ tự do của nó bị hạn chế, làm giảm nguy cơ quá khớp (nhưng làm tăng nguy cơ thiếu khớp - underfitting).

Để tránh hiện tượng quá khớp dữ liệu huấn luyện, bạn cần hạn chế sự tự do của cây quyết định trong quá trình huấn luyện. Như bạn đã biết, điều này được gọi là chuẩn hóa (regularization). Các siêu tham số chuẩn hóa phụ thuộc vào thuật toán được sử dụng, nhưng nhìn chung bạn có thể hạn chế độ sâu tối đa của cây quyết định. Trong Scikit-Learn, điều này được kiểm soát bởi siêu tham số `max_depth`. Giá trị mặc định là `None`, có nghĩa là không giới hạn.

Việc giảm `max_depth` sẽ giúp điều chỉnh mô hình và do đó giảm nguy cơ quá khớp.

Lớp `DecisionTreeClassifier` có một vài tham số khác cũng giới hạn hình dạng của Cây quyết định:

- `max_features`: số lượng tối đa các đặc trưng được đánh giá để phân tách tại mỗi nút.
- `max_leaf_nodes`: số lượng nút lá tối đa
- `min_samples_split`: số lượng mẫu tối thiểu mà một nút phải có trước khi nó có thể được tách.
- `min_samples_leaf`: số lượng mẫu tối thiểu mà một nút lá phải có để được tạo.
- `min_weight_fraction_leaf`: tương tự như `min_samples_leaf` nhưng được biểu thị dưới dạng phân số của tổng số trường hợp được trọng số hóa.

Việc tăng các siêu tham số `min_*` hoặc giảm các siêu tham số `max_*` sẽ điều chỉnh mô hình.

GHI CHÚ

Các thuật toán khác hoạt động bằng cách đầu tiên huấn luyện Cây Quyết định mà không có bất kỳ ràng buộc nào, sau đó cắt tỉa (xóa) các nút không cần thiết. Một nút mà tất cả các con của nó đều là nút lá được coi là không cần thiết nếu sự cải thiện về độ tinh khiết mà nó mang lại không có ý nghĩa thống kê. Các kiểm định thống kê tiêu chuẩn, chẳng hạn như kiểm định χ^2 (kiểm định chi bình phương), được sử dụng để ước tính xác suất rằng sự cải thiện hoàn toàn là kết quả của sự ngẫu nhiên (được gọi là giả thuyết không). Nếu xác suất này, được gọi là giá trị p, cao hơn một ngưỡng nhất định (thường là 5%, được kiểm soát bởi một siêu tham số), thì nút đó được coi là không cần thiết và các con của nó bị xóa. Quá trình cắt tỉa tiếp tục cho đến khi tất cả các nút không cần thiết đã được cắt tỉa.

Hãy thử nghiệm phương pháp điều chỉnh tham số trên tập dữ liệu moons, được giới thiệu trong [Chương 5](#). Chúng ta sẽ huấn luyện một cây quyết định không sử dụng phương pháp điều chỉnh tham số, và một cây khác với `min_samples_leaf=5`.

[Hình 6-3](#) thể hiện ranh giới quyết định của mỗi cây.

```
from sklearn.datasets import make_moons
```

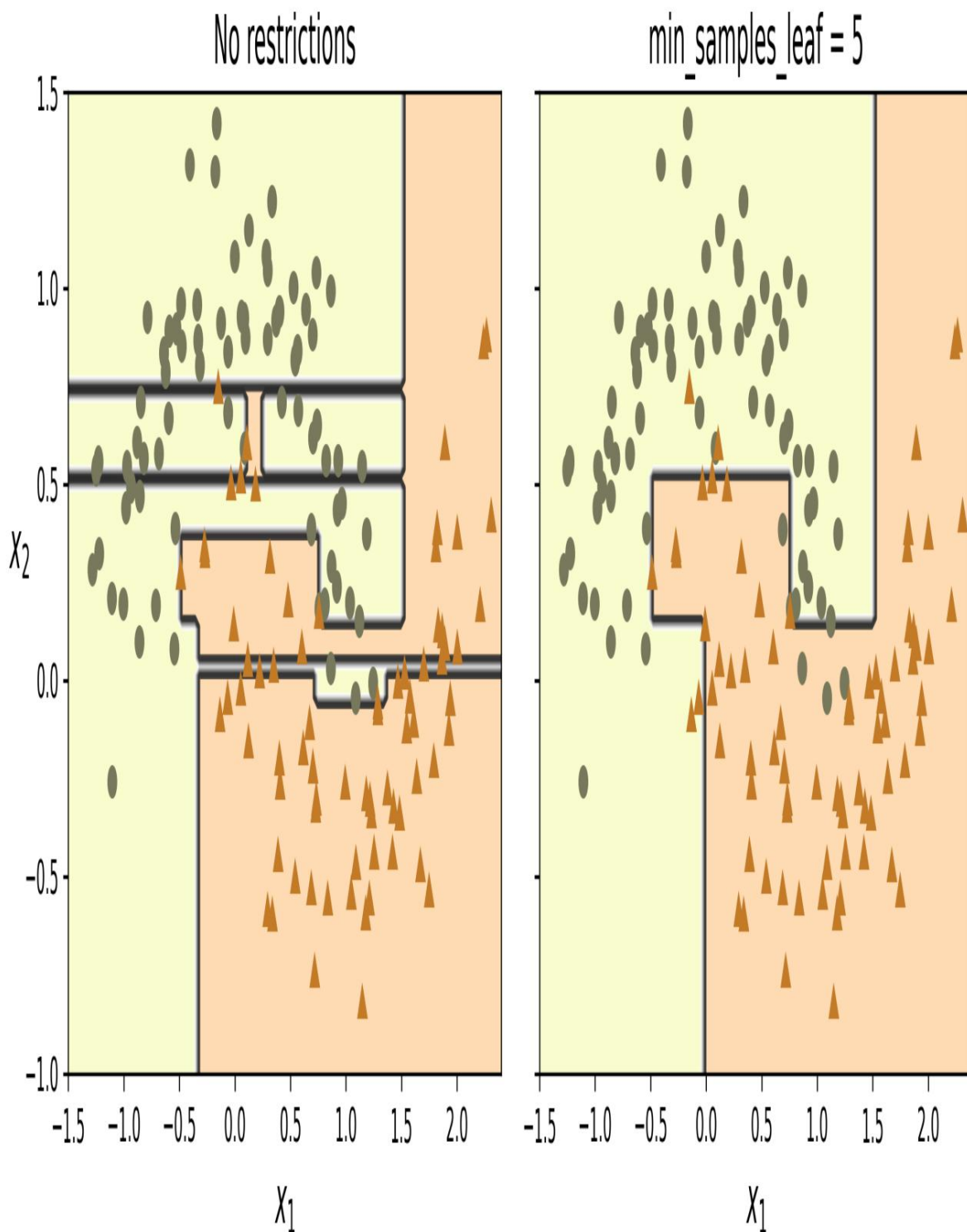
```
X_moons, y_moons = make_moons(n_samples=150, noise=0.2, random_state=42)
```

```
tree_clf1 = DecisionTreeClassifier(random_state=42)
```

```
tree_clf2 = DecisionTreeClassifier(min_samples_leaf=5, random_state=42)
```

```
tree_clf1.fit(X_moons, y_moons)
```

```
tree_clf2.fit(X_moons, y_moons)
```



Hình 6-3. Đường ranh giới quyết định của cây chưa được chuẩn hóa (bên trái) và cây đã được chuẩn hóa (bên phải)

Mô hình không được điều chỉnh ở bên trái rõ ràng là bị quá khớp, và mô hình được điều chỉnh ở bên phải có thể sẽ tổng quát hóa tốt hơn. Chúng ta có thể kiểm chứng điều này bằng cách đánh giá cả hai cây trên một tập dữ liệu thử nghiệm được tạo ra bằng cách sử dụng một hạt giống ngẫu nhiên khác:


```
>>> X_moons_test, y_moons_test = make_moons(n_samples=1000, noise=0.2, random_state=43)
...
...
>>> tree_clf1.score(X_moons_test, y_moons_test) 0.898

>>> tree_clf2.score(X_moons_test, y_moons_test) 0.92
```

Thực tế, cây thứ hai có độ chính xác tốt hơn trên tập dữ liệu thử nghiệm.

Hồi quy

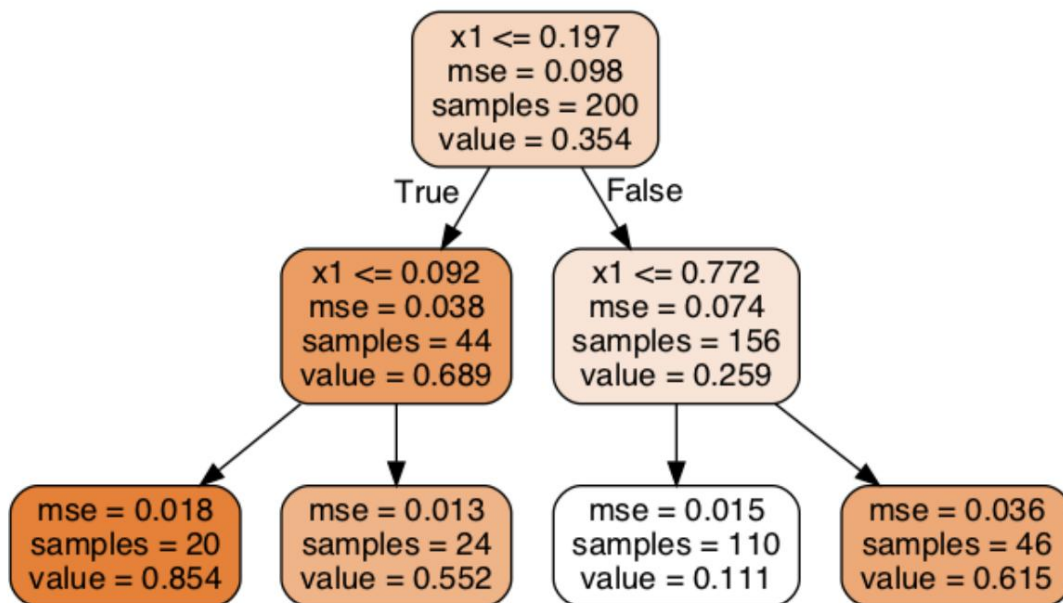
Cây quyết định cũng có khả năng thực hiện các tác vụ hồi quy. Hãy cùng xây dựng một cây hồi quy bằng cách sử dụng lớp `DecisionTreeRegressor` của Scikit-Learn, huấn luyện nó trên một tập dữ liệu bậc hai nhiều với `max_depth=2`:

```
import numpy as np from
sklearn.tree import DecisionTreeRegressor

np.random.seed(42)
X_quad = np.random.rand(200, 1) - 0.5 # một đặc trưng đầu vào ngẫu nhiên duy nhất y_quad =
X_quad ** 2 * np.random.rand(200, 1)

tree_reg = DecisionTreeRegressor(max_depth=2, random_state=42) tree_reg.fit(X_quad,
y_quad)
```

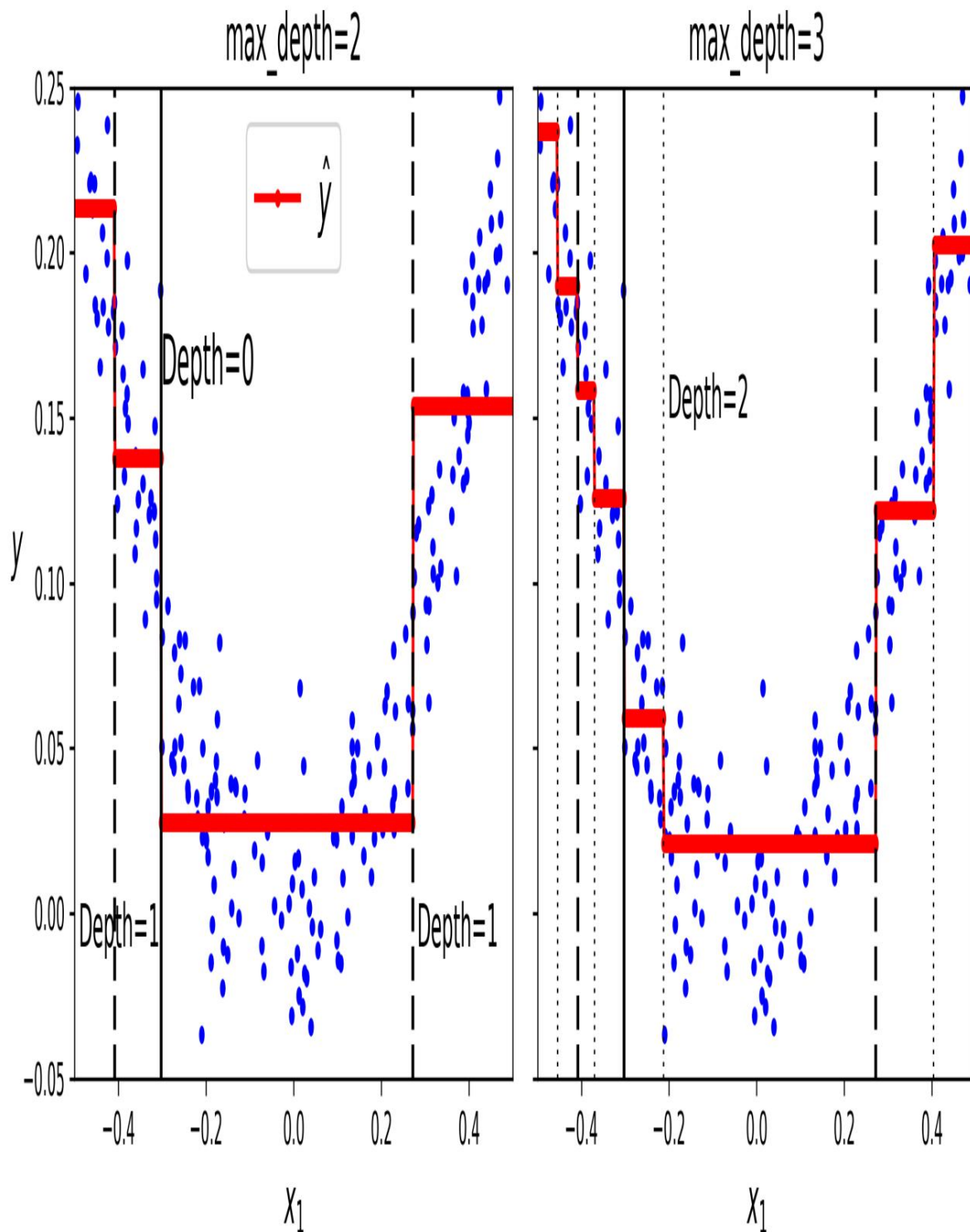
Cây kết quả được thể hiện trong Hình 6-4.



Hình 6-4. Cây quyết định cho hồi quy

Cây này trông rất giống với cây phân loại mà bạn đã xây dựng trước đó. Sự khác biệt chính là thay vì dự đoán một lớp trong mỗi nút, nó dự đoán một giá trị. Ví dụ, giả sử bạn muốn đưa ra dự đoán cho một trường hợp mới với $x = 0,2$. Nút gốc hỏi liệu $x \leq 0,197$ hay không. Vì không, thuật toán chuyển sang nút con bên phải, nút này hỏi liệu $x \leq 0,772$ hay không. Vì có, thuật toán chuyển sang nút con bên trái. Đây là một nút lá, và nó dự đoán giá trị $= 0,111$. Dự đoán này là giá trị mục tiêu trung bình của 110 trường hợp huấn luyện liên quan đến nút lá này, và nó dẫn đến sai số bình phương trung bình bằng 0,015 trên 110 trường hợp này.

Các dự đoán của mô hình này được thể hiện ở bên trái trong **Hình 6-5**. Nếu bạn đặt `max_depth=3`, bạn sẽ nhận được các dự đoán được thể hiện ở bên phải. Hãy chú ý cách giá trị dự đoán cho mỗi vùng luôn là giá trị mục tiêu trung bình của các trường hợp trong vùng đó. Thuật toán chia mỗi vùng theo cách làm cho hầu hết các trường hợp huấn luyện càng gần với giá trị dự đoán đó càng tốt.



Hình 6-5. Dự đoán của hai mô hình hồi quy cây quyết định

Thuật toán CART hoạt động về cơ bản giống như trước đây, ngoại trừ việc thay vì cố gắng chia tập huấn luyện theo cách giảm thiểu độ không thuần khiết, giờ đây nó cố gắng chia tập huấn luyện theo cách giảm thiểu MSE. Phương trình 6-4 thể hiện hàm chi phí mà thuật toán cố gắng giảm thiểu.

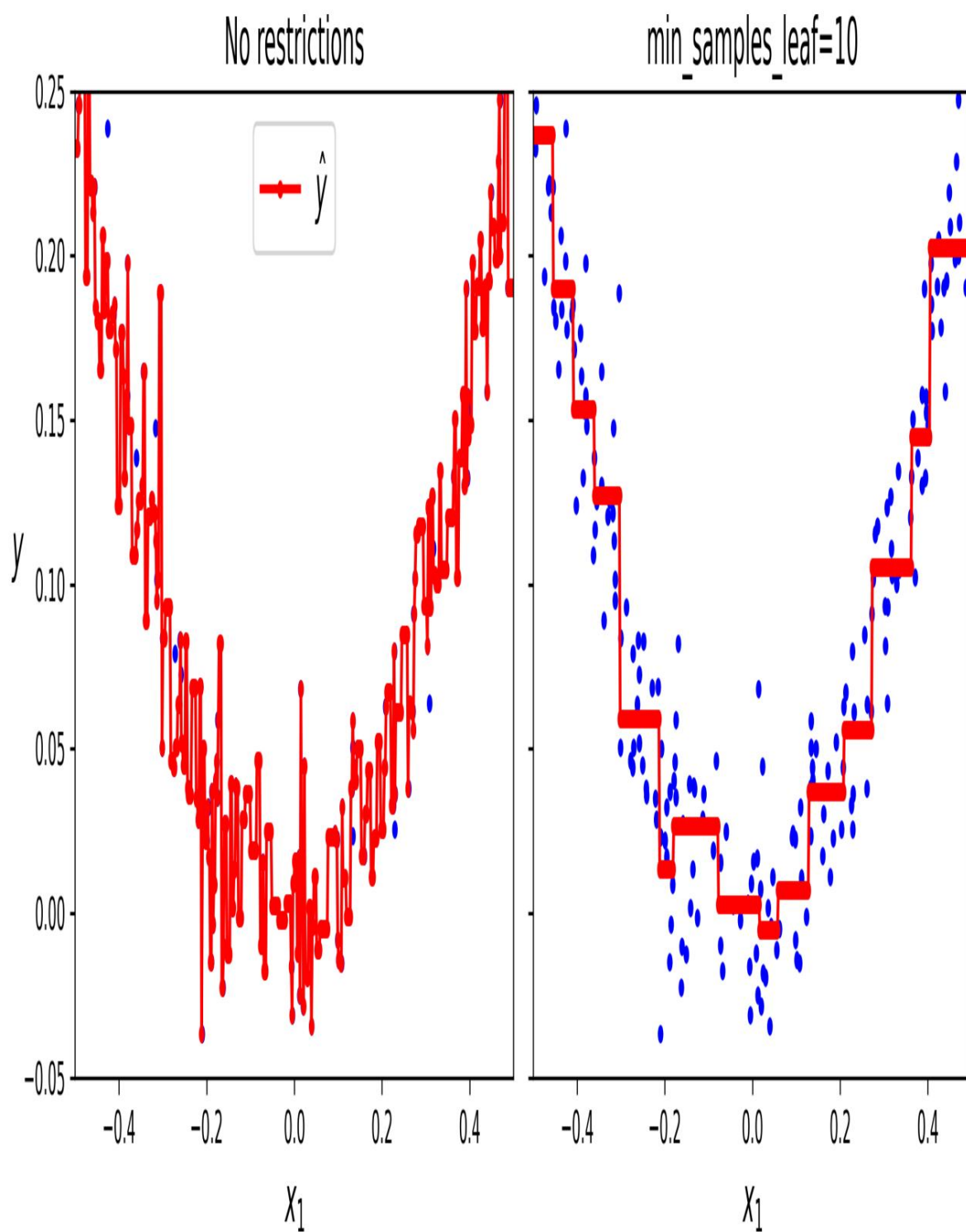
Phương trình 6-4. Hàm chi phí CART cho hồi quy

$$J(k, t_k) = \frac{m_{\text{left}}}{m} \text{MSE}_{\text{left}} + \frac{m_{\text{right}}}{m} \text{MSE}_{\text{right}} \text{ where}$$

$$\text{MSE}_{\text{node}} = \frac{\sum_{i \in \text{node}} (y_i - \hat{y}_{\text{node}})^2}{m_{\text{node}}}$$

$$\hat{y}_{\text{node}} = \frac{\sum_{i \in \text{node}} y_i}{m_{\text{node}}}$$

Cũng giống như đối với các bài toán phân loại, Cây quyết định dễ bị quá khớp khi xử lý các bài toán hồi quy. Nếu không có bất kỳ sự điều chỉnh nào (tức là sử dụng các siêu tham số mặc định), bạn sẽ nhận được các dự đoán ở bên trái trong **Hình 6-6**. Rõ ràng là các dự đoán này đang quá khớp với tập dữ liệu huấn luyện một cách nghiêm trọng. Chỉ cần đặt `min_samples_leaf=10` sẽ dẫn đến một mô hình hợp lý hơn nhiều, được thể hiện ở bên phải trong **Hình 6-6**.



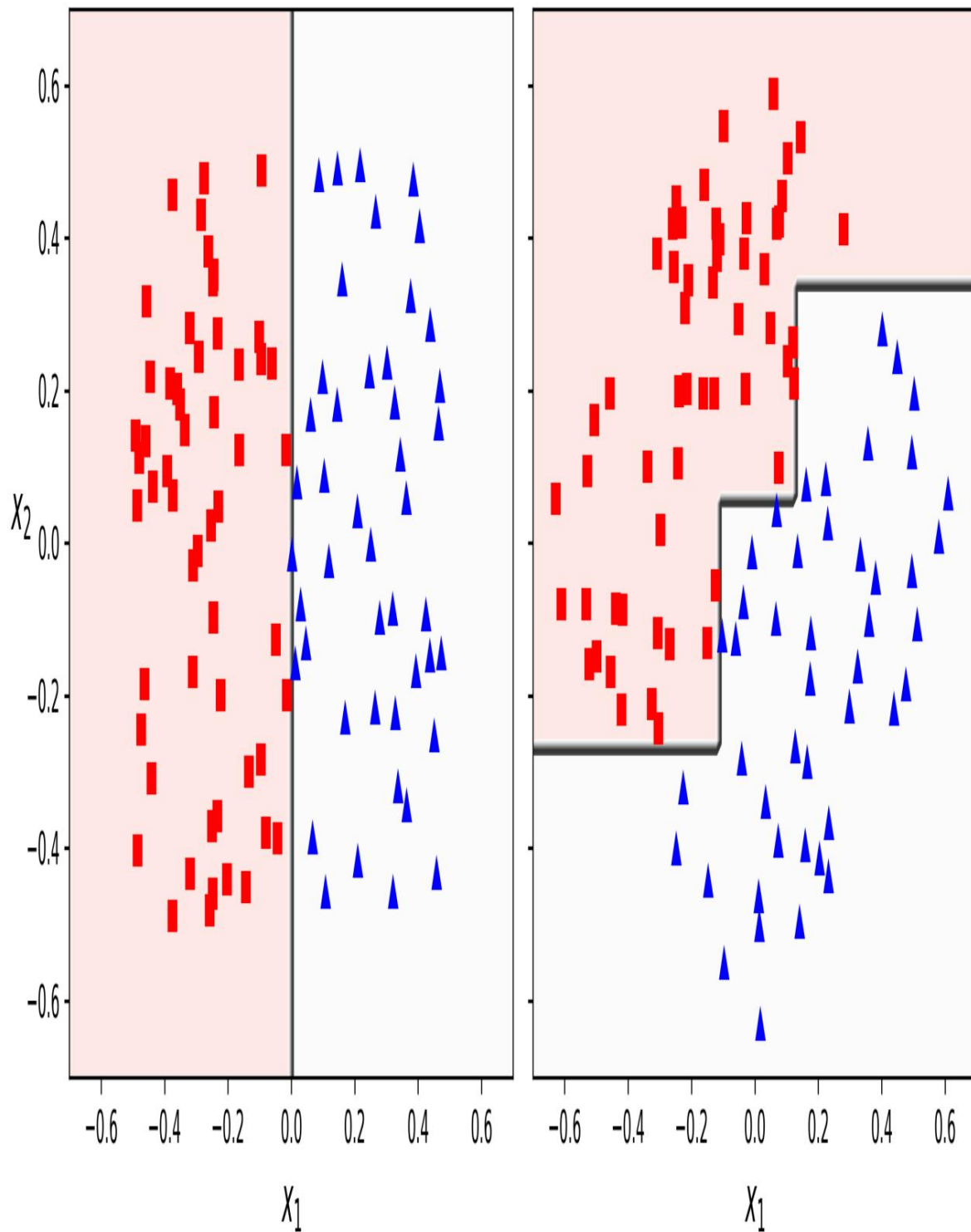
Hình 6-6. Dự đoán của cây hồi quy không được điều chỉnh (bên trái) và cây được điều chỉnh (bên phải)

Độ nhạy cảm với hướng trục

Hy vọng đến giờ bạn đã tin rằng cây quyết định có rất nhiều ưu điểm: chúng tương đối dễ hiểu và diễn giải, dễ sử dụng, linh hoạt và mạnh mẽ.

Tuy nhiên, chúng cũng có một vài hạn chế. Thứ nhất, như bạn có thể nhận thấy, Cây quyết định rất thích các ranh giới quyết định vuông góc (tất cả các điểm phân chia đều vuông góc với một trục), điều này khiến chúng nhạy cảm với hướng của dữ liệu. Ví dụ, **Hình 6-7** cho thấy một tập dữ liệu có thể phân tách tuyến tính đơn giản: ở bên trái, Cây quyết định có thể dễ dàng phân chia nó, trong khi ở bên phải, sau khi tập dữ liệu được xoay 45° , ranh giới quyết định trông phức tạp một cách không cần thiết.

Mặc dù cả hai cây quyết định đều phù hợp hoàn hảo với tập dữ liệu huấn luyện, nhưng rất có thể mô hình bên phải sẽ không có khả năng khái quát hóa tốt.



Hình 6-7. Độ nhạy cảm với phép xoay tập dữ liệu huấn luyện

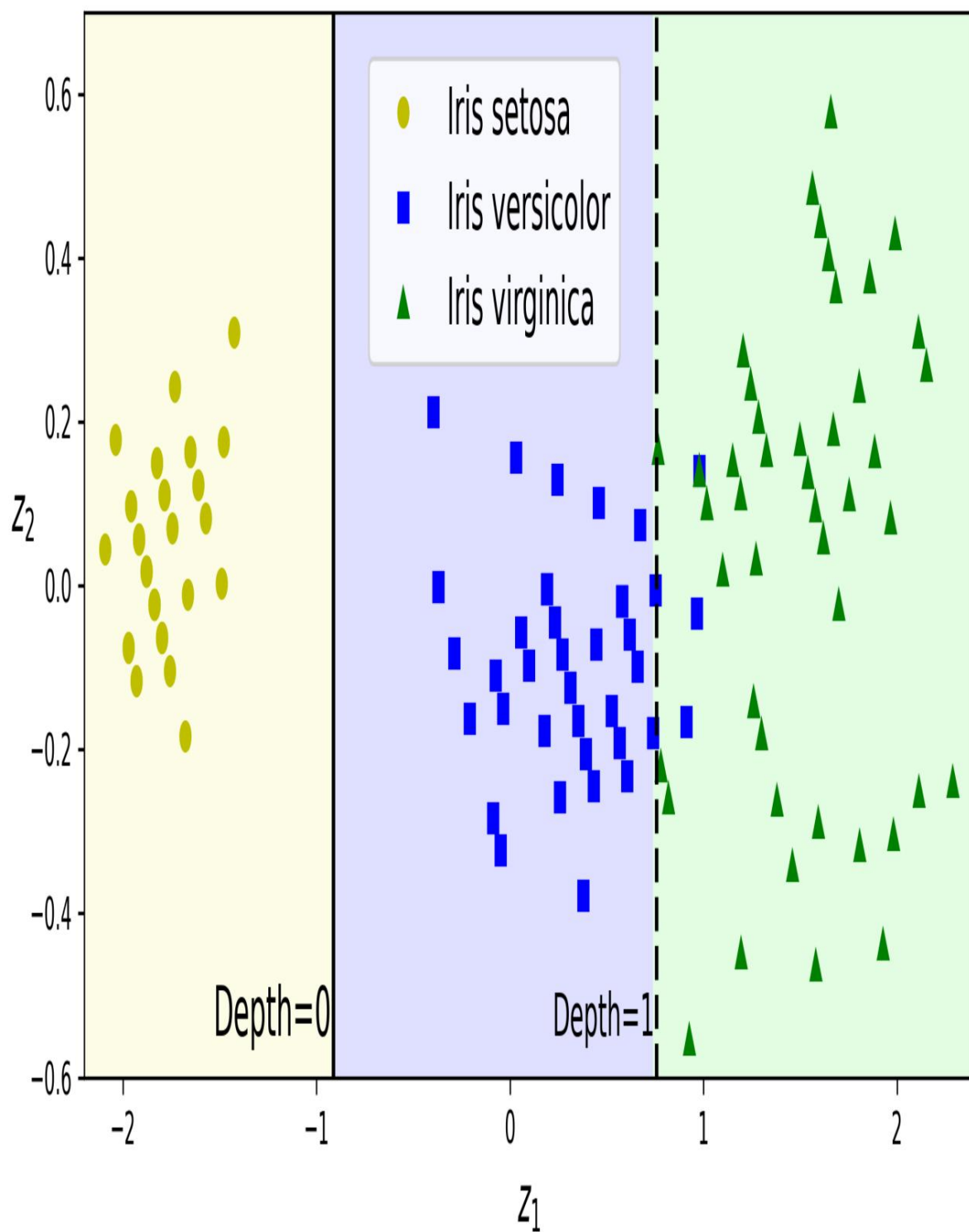
Một cách để hạn chế vấn đề này là chuẩn hóa dữ liệu rồi áp dụng phép biến đổi Phân tích Thành phần Chính (PCA). Chúng ta sẽ xem xét PCA chi tiết trong [Chương 8](#), nhưng hiện tại bạn chỉ cần biết rằng nó xoay dữ liệu theo cách làm giảm sự tương quan giữa các đặc trưng, điều này thường (không phải lúc nào cũng vậy) giúp việc xây dựng cây quyết định dễ dàng hơn. Hãy tạo một tập dữ liệu nhỏ.

Chúng ta sẽ sử dụng PCA để xử lý dữ liệu, sau đó huấn luyện một `DecisionTreeClassifier` trên dữ liệu đó. **Hình 6-8** hiển thị các ranh giới quyết định của cây đó: như bạn thấy, phép xoay giúp phù hợp với tập dữ liệu khá tốt chỉ bằng một đặc trưng duy nhất, đó là chiều rộng z .

1, đó là một hàm tuyến tính của chiều dài cánh hoa ban đầu và

```
from sklearn.decomposition import PCA from
sklearn.pipeline import make_pipeline from
sklearn.preprocessing import StandardScaler

pca_pipeline = make_pipeline(StandardScaler(), PCA())
X_iris_rotated = pca_pipeline.fit_transform(X_iris)
tree_clf_pca = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf_pca.fit(X_iris_rotated, y_iris)
```

Hình 6-8. Ranh giới quyết định của cây trên tập dữ liệu iris đã được thu nhỏ và xoay bằng PCA.

Cây quyết định có độ biến thiên cao

Nhìn chung, vấn đề chính của Cây Quyết định là chúng có độ biến thiên khá cao: những thay đổi nhỏ đối với các siêu tham số hoặc dữ liệu có thể tạo ra các mô hình rất khác nhau. Trên thực tế, vì thuật toán huấn luyện được Scikit-Learn sử dụng là ngẫu nhiên—nó chọn ngẫu nhiên tập hợp các đặc trưng để đánh giá tại mỗi nút—ngay cả khi huấn luyện lại cùng một Cây Quyết định trên cùng một tập dữ liệu cũng có thể tạo ra một mô hình rất khác, chẳng hạn như mô hình được thể hiện trong **Hình 6-9** (trừ khi bạn đặt siêu tham số `random_state`). Như bạn có thể thấy, nó trông rất khác so với Cây Quyết định trước đó (**Hình 6-2**).